XIMOD Architect — XML Builder

用户手册

专为您的模组设计的XML 安装程序。

Skyrim · Fallout · Starfield · Palworld · 及更多游戏

Version 2.2.2

跨平台：Windows · Linux · macOS



目录

1. 概述 6

1.1 什么是 XIMOD Architect？ 6

1.2 FOMOD 格式 6

1.3 安装与系统要求 7

Windows 7

Linux 7

macOS 7

1.4 文件结构 7

2. 入门 9

2.1 启动应用 9

2.2 界面概览 9

2.3 创建新项目 10

2.4 打开和保存项目 10

2.5 首次启动 10

2.6 自由窗口与键盘导航 10

3. 菜单 12

3.1 文件菜单 12

3.2 编辑菜单 13

3.3 项目菜单 13

3.4 工具菜单 14

3.5 帮助菜单 15

4. 功能详解 16

4.1 模组信息标签页 16

4.2 安装步骤标签页 16

组与选择类型 17

选项与默认类型 17

插件文件 17

组 / 页面目标 18

条件标志 18

4.3 必需安装项标签页 19
4.4 条件安装项标签页 19
4.5 “设置”窗口 19
4.6 保存前脚本与保存后脚本 20
示例 —— 保存前脚本（Windows） 21
示例 —— 保存后脚本（Windows） 21
Linux / macOS 等效写法 21
4.7 “关于”窗口 21
4.8 生成 FOMOD 文件 21
4.9 翻译编辑器 22
4.10 XML 编辑器 24
4.11 字体管理 24
4.12 合并两个 FOMOD 25
4.13 导出分发存档 25
4.14 “属性”窗口 25
4.15 命令行模式（CLI） 27
4.16 检查更新 27
5. FOMOD 标签参考 28
5.1 info.xml 文件 28
5.2 ModuleConfig.xml 文件 28
根与页眉 28
文件与文件夹 28
步骤与组 29
选项（plugins） 29
依赖与条件 30
6. ModuleConfig.xml 文件详解 31
6.1 安装程序如何读取文件 31
6.2 根与页眉 31
6.3 安装文件 31
6.4 步骤、组与选项 32

6.5 选项的类型及其行为 33
6.6 标志、依赖与条件安装 33
7. ModuleConfig.xml 示例 35
7.1 最简示例 —— 必装文件 35
7.2 互斥选择 —— SelectExactlyOne 35
7.3 条件安装 —— 标志 35
7.4 一个页面，多个步骤 —— SelectAny + 标志 36
7.5 带可自定义默认值的预设 38
7.6 欢迎页与结束页 41
8. 版本 2 —— 新功能 43
8.1 目标冲突检测 43
8.2 7z 导出 43
8.3 可复用模板 43
8.4 从文件夹新建 43
8.5 拖放式文件分配 43
8.6 FOMOD 比较 43
8.7 可视化条件编辑器 44
8.8 图像检查与优化 44
8.9 命令行新增项 44
8.10 后台导出与验证 44
8.11 崩溃恢复 44
8.12 更清晰的界面 44
8.13 翻译现有的 FOMOD 44
8.14 打开模组存档 46
8.15 导入保真度 46
8.16 备份与恢复 46
8.17 将主文件作为自动条件 46
8.18 轻量插件（ESL）与插件报告 47
8.19 更丰富的模拟器与场景 47
8.20 每个选项的安装大小 47

8.21 Nexus 描述生成器 48
8.22 可编辑的条件编辑器 48
8.23 项目字符串表格 48
8.24 读取 BSA 和 BA2 存档 48
8.25 嵌套条件组与版本条件 49
8.26 编辑游戏及其分类 50
8.27 树颜色 50
8.28 文件对话框会记住您上次的位置 50
8.29 预设向导 50
8.30 在 FOMOD 之间剪切、复制和粘贴 51
8.31 在磁盘上重命名来源 52
8.32 查找缺失文件 52
附录 52
附录 A —— Config.ini 文件 52
附录 B —— 键盘快捷键 53
附录 C —— 术语表 54
附录 D —— 完整 FOMOD 示例 55
D.1 info.xml 55
D.2 ModuleConfig.xml 55
附录 E —— 许可证与致谢 59

1. 概述

1.1 什么是 XIMOD Architect?

XIMOD Architect 是一款跨平台工具，用于为游戏模组创建 FOMOD 安装程序——Skyrim、Fallout、Starfield 以及 Vortex 或 Mod Organizer 2 能够管理的任何其他游戏。它让您以可视化方式构建完整的安装向导——步骤、选项组、选项、条件文件——然后自动生成模组管理器能够识别的标准化 XML 文件。

FOMOD 是一种模组管理器格式，而非某款游戏专用的格式：您构建的安装程序适用于所有 Bethesda 游戏，以及 Vortex 或 Mod Organizer 2 能够管理的大多数其他游戏——只要一款游戏可以用 Vortex 安装模组，它就可以使用 FOMOD 安装程序。Categories.json 文件中的内置列表（十一款 Bethesda 游戏：Skyrim 的三个版本——Legendary、Special Edition 和 VR——、Fallout 3、Fallout: New Vegas、Fallout 4 和 Fallout 4 VR、Oblivion 和 Oblivion Remastered、Morrowind 以及 Starfield）仅用于填写 info.xml 中的游戏和 Nexus 分类；它并不限制安装程序可在何处使用。该列表是外部文件且可编辑：工具 → 游戏 / 分类…（§ 8.26）可添加任何其他游戏，而无需重新编译应用。

本软件是对原 FOMOD Creation Tool 用 Rust 语言进行的重写。此次重构带来了更佳的性能、在 Windows、Linux 和 macOS 上一致的行为，以及现代化的界面。XIMOD Architect 还以其十分先进的国际化系统著称：界面可翻译为大量语言，并能动态加载适配各书写系统的字体。

主要功能如下：

- 创建完整的 FOMOD 安装包；
- 多步骤安装向导；
- 具有不同选择类型（恰好一个、至少一个、任意等）的选项组；
- 根据用户选择进行的条件文件安装；
- 驱动选项类型的依赖模式；
- 控制安装流程的“标志”（条件标志）系统；
- 在保存前或保存后运行的自定义脚本；
- 多语言界面与动态字体；
- 原生的透明启动画面。

1.2 FOMOD 格式

FOMOD（“Fallout Mod”）是 Bethesda 游戏模组安装程序的标准格式，Vortex 和 Mod Organizer 2 等模组管理器也将其用于大多数其他游戏。一个 FOMOD 安装程序基于集中在名为 fomod 的子文件夹中的两个 XML 文件：

- info.xml —— 模组元数据：名称、作者、版本、网站、描述、分类。
- ModuleConfig.xml —— 安装配置：步骤、组、选项、文件、条件。

XIMOD Architect 生成的文件与主流模组管理器兼容：

- Mod Organizer 2（MO2）；
- Vortex；

- Nexus Mod Manager（NMM）。

> 注意 —— fomod 文件夹的名称以及 info.xml 和 ModuleConfig.xml
> 这两个名称由规范强制规定，模组管理器会按原样识别它们。切勿对其重命名。

1.3 安装与系统要求

系统要求（仅供参考）：

项目	建议
系统	Windows 10 或更高版本、Linux（较新的 glibc）或 macOS 11 或更高版本。
显示	兼容 OpenGL 3.3 的显卡（egui 的 “glow” 渲染器）。
内存	数百 MB —— 本应用十分轻量。
磁盘空间	数十 MB，具体取决于 Noto 字体以及 assets/ 中提供的旗帜。

Windows

将可执行文件 ximod-architect.exe、assets 文件夹（data、locales、fonts、images、icons）以及可选的用于启动画面的 assets/images/splash.png 文件放在同一文件夹中。应用图标在编译时已嵌入可执行文件。然后运行 ximod-architect.exe。

Linux

packaging/linux 中提供了一个安装脚本：

```
# System-wide installation (requires sudo)
sudo ./packaging/linux/install.sh

# Installation for the current user
./packaging/linux/install.sh --user
```

从源代码构建需要 Rust 1.99.0 或更高版本，并且在 Linux 上需要以下开发库：libgtk-3-dev、libxcb-render0-dev、libxcb-shape0-dev、libxcb-xfixes0-dev、libspeechd-dev、libxkbcommon-dev、libssl-dev 和 libx11-dev。

macOS

有一个脚本可创建可直接分发的应用程序包（.app）：

```
./packaging/macos/create-bundle.sh
```

“XIMOD Architect.app” 应用包将生成在 dist 文件夹中。随后可使用 hdiutil 工具制作磁盘映像（.dmg）。

1.4 文件结构

随可执行文件附带的文件树如下：

```
ximod-architect(.exe)            → executable
Config.ini                       → configuration (created on first launch)
assets/
   data/
      Categories.json            → games and Nexus categories
      Languages.json             → 6,777 languages (endonym, ISO codes, font)
      Countries.json             → 244 countries (names, flag, official languages)
      CountryLanguages.json      → languages spoken by country
   locales/                      → translations (one folder per language)
      eng/main.ftl
      fra/main.ftl
      …                          → 32 languages provided
   fonts/                        → Noto fonts (one per writing system)
   icons/                        → application icons
   images/
      splash.png                 → splash screen
      svg/                       → country flags (244 SVG files)
```

2. 入门

2.1 启动应用

启动时，XIMOD Architect
首先显示一个原生的透明启动画面，带有淡入淡出效果，其持续时间可调（默认两秒；该画面可被禁用）。随后主窗口打开，居中显示在主显示器上。其初始大小来自配置文件（默认 1280 × 800），且不能小于 800 × 600。标题栏显示应用名称及其版本号。

2.2 界面概览

主窗口由五个区域组成：

- 顶部的菜单栏——文件、编辑、项目、工具和帮助——
  其正下方是包含日常操作（新建、打开、保存、撤销、重做、验证、预览、导出）的工具栏；参见第 3 章；
- 左侧的项目树：模组信息，然后是各安装步骤及其选项组和选项、必装文件以及条件安装。单击某个节点可编辑它，右键单击可调出其操作（添加、复制、移动、删除、另存为模板），并可将一个选项拖到另一个组上、一个组拖到另一个步骤上，或将一个步骤拖到另一个步骤之前以移动它；“安装步骤”和“条件安装”旁边的 + 按钮可直接添加元素；
- 居中的检查器，显示所选节点的编辑器：模组信息、一个步骤（名称、可见性条件、页面目标、其组的列表）、一个组（名称、选择类型、组目标、其选项的列表）、一个选项（描述、图像、类型、标志、依赖模式、文件）、必装文件或某个条件集；检查器顶部的面包屑导航让您可返回上一级；
- 底部的“问题”面板，在执行 项目 → 验证 FOMOD
  后显示：每条结果都会连同其严重程度一并列出，单击某条即可在树中选中相关节点；树本身会在相关节点旁显示一个小的错误/警告计数器；
- 最底部的状态栏：左侧显示最新消息（启动时为“就绪”）；右侧显示步骤和选项的数量，当项目有未保存的更改时，还会显示一个圆点后跟“已修改”。成功、警告和错误也会以通知形式出现在右上角。

界面外观（深色、浅色或跟随系统主题、字体大小、语言）在 § 4.5 所述的“设置”窗口中设定。

在树中，单击可选中一个节点并在检查器中显示它；在步骤或组上双击的效果相同，并会将其展开。步骤的右键菜单提供展开（全部、选中、从选中开始——
该步骤及其下的所有内容）和折叠（同样三种选择），组的菜单提供展开和折叠（全部、选中），位于常用的添加 / 复制 / 移动 / 删除命令之前。每个标签页都保有各自的“问题”面板：所显示的结果始终属于活动标签页中加载的
FOMOD，在某个标签页中启动的验证会在该标签页中完成，即使您在此期间切换到了另一个标签页。

当没有文本框获得焦点时，也可用键盘操控树：↑ 和 ↓
使光标环在各行之间移动（起始于所选节点），Enter 选中其下的行，→
展开一个已折叠的步骤或组（对于已展开的，则移至其第一个子项），←
折叠一个已展开的项（对于已折叠的行或某个选项，则返回其父项）。光标环移动时会自动滚动进入视野。

2.3 创建新项目

创建安装程序通常遵循以下步骤：

- 新建项目 —— 文件菜单 → 新建，然后选择包含模组文件的根目录（“模组信息”标签页中的“浏览…”按钮）。
- 填写信息 —— 在“模组信息”标签页中输入模组名称、作者、版本、游戏、分类、网站、页眉图像和描述。
- 构建向导 —— 在“安装步骤”标签页中添加步骤，然后添加选项组，再添加选项。
- 关联文件 —— 为每个选项添加要安装的文件或文件夹，如有需要再在表格中设置其目标路径（默认为空，表示安装到游戏 Data 文件夹的根目录）。
- 定义条件 —— 使用标志和条件安装，使安装适应用户的选择。
- 保存 —— 文件菜单 → 保存（Ctrl+S）：XIMOD Architect 会创建 fomod 文件夹并将 info.xml 和 ModuleConfig.xml 写入其中。

2.4 打开和保存项目

保存操作始终会在根目录下生成 fomod 子文件夹，然后以 UTF-8 将两个 XML 文件写入其中。在未定义根目录之前，“保存”命令始终处于禁用状态。

提供两种打开方式。“打开文件夹…”会询问根目录，并从其 fomod 子文件夹加载项目。“打开文件…”则直接询问某个 XML 文件：随后会推断出根目录（假定该文件位于某个 fomod 子文件夹中）。读取时，XIMOD Architect 会自动识别文件编码（带或不带字节顺序标记的 UTF-8，以及 UTF-16），从而可以重新打开由其他工具生成的安装程序。

每个成功打开的项目都会添加到最近文件列表的顶部，可通过 文件 → 最近访问，并可在“设置”的“最近文件”标签页中管理。

2.5 首次启动

在首次启动时，XIMOD Architect 会以英语启动并自动打开“设置”窗口，以便用户选择其国家/地区，然后选择其语言。

此行为由 Config.ini 文件中的 FirstStart 键驱动：只要初始配置尚未保存，它就为 0，而一旦首次单击“保存”，它就切换为 1。此后的启动将直接以所选语言开始。

如果用户以“取消”关闭该窗口，该键将保持为 0，窗口会在下次启动时重新打开。

2.6 自由窗口与键盘导航

有几个工具会以自由（独立）窗口打开：“国家/地区选择”选择器、“属性”窗口、翻译编辑器和 XML 编辑器。每个窗口都可自由移动和调整大小，包括在第二块屏幕上，并与主窗口并行工作。

居中与持久化 —— 首次打开时，自由窗口会居中于 XIMOD 主窗口。如果用户移动或调整其大小，则在关闭时（以及应用退出时）会记住其位置和大小，并在下次打开时恢复。这些值保存在 Config.ini 的 [WindowPositions] 节中（参见附录 A）。主窗口本身会记住其大小但不记住其位置：每次启动时都保持居中。

用 Esc 关闭 —— Esc 键可关闭活动的自由窗口以及主窗口的模态对话框（设置、关于、确认、验证报告、脚本）；对于确认对话框，Esc 的作用是“取消”，绝不会执行相应操作。两个例外：翻译编辑器（以免在输入时意外关闭它）和 XIMOD 主窗口不会因 Esc 而关闭。

键盘导航 ——

“属性”窗口中的列表、翻译编辑器表格以及国家/地区选择器的旗帜网格，均可完全通过键盘操作：方向键（以及数字小键盘）、Page Up/Page Down、Home/End、Enter 和 Tab/Shift+Tab。各窗口专有的细节在相关章节中描述，并在附录 B 中汇总。

清除筛选 ——

每个“筛选：”字段的右侧都有一个清除图标（红色叉号），单击即可清空筛选条件。

3. 菜单

菜单栏包含五个菜单：文件、编辑、项目、工具和帮助。其下方有一个工具栏，可一键访问日常操作（新建、打开、保存、验证、预览、导出）。所示快捷键在全局范围内同样有效（Ctrl 键在 macOS 上对应 Cmd）；仅在模态对话框打开时它们才被忽略。模态对话框会使窗口其余部分变暗；Enter 触发其主按钮，Esc 取消。

3.1 文件菜单

命令	快捷键	描述
新建	Ctrl+N	在新标签页中打开一个新的空白项目（仅当当前标签页仍为全新状态时才会被替换）。
从文件夹新建…	—	扫描一个模组文件夹，并提出一个框架，其中每个子文件夹成为一个选项。参见 § 8.4。
打开文件夹…	Ctrl+O	选择一个根目录，并从其 fomod 子文件夹加载项目。
打开文件…	Ctrl+Shift+O	直接选择一个 info.xml 或 ModuleConfig.xml；根目录将被推断。不在 fomod 文件夹内的文件会被拒绝。
打开压缩包…	—	将一个 .zip 或 .7z 模组存档（如从 Nexus 下载的）解压到配置目录的工作文件夹中，然后打开其 FOMOD。已解压过的存档会被重用。.rar 存档会被拒绝：请先用 7-Zip 解压。将存档拖放到窗口上效果相同。参见 § 8.14。
保存	Ctrl+S	生成 fomod/info.xml 和 fomod/ModuleConfig.xml。如果项目尚无根目录，会先询问一个文件夹。
另存为…	Ctrl+Shift+S	为项目选择一个（新的）根目录，然后将其保存到那里。
恢复备份…	—	列出保存在 fomod/backups 下的 FOMOD XML 文件的历史版本，并显示其与当前项目之间的差异数量；每个版本都可进行比较或恢复到编辑器中。当项目尚无备份时禁用。参见 § 8.16。
合并 FOMOD…	—	将另一个 FOMOD 的步骤、必装文件和条件安装添加到当前项目。只要未定义根目录便禁用。参见 § 4.12。
导出分发存档…	—	写入 FOMOD，然后在后台将整个根目录压缩为可直接发布的 .zip 或 .7z 存档，并带有进度窗口。参见 § 4.13 和 § 8.10。
关闭 FOMOD	—	关闭活动标签页，若其有未保存的更改会先行询问。

命令	快捷键	描述
关闭所有 FOMOD	—	关闭每个标签页，若其中任何一个有未保存的更改会先行询问。
最近	—	列出最近打开的项目的子菜单。如果列表为空，则显示“（空）”。
退出	Ctrl+Q	关闭应用。未保存的项目会被列出并可先行保存；配置在退出时保存。

3.2 编辑菜单

命令	快捷键	描述
撤销	Ctrl+Z	撤销对项目的上一次更改（同一字段中的连续按键会被合并）。无可撤销内容时禁用。
重做	Ctrl+Y	重新应用上一次被撤销的更改（Ctrl+Shift+Z 同样有效）。
剪切	Ctrl+X	移除树中所选的步骤、组或选项，并将其保留在剪贴板中。编辑 → 撤销 可将其恢复。参见 § 8.30。
复制	Ctrl+C	将所选的步骤、组或选项连同其包含的全部内容复制到剪贴板——同时也复制到系统剪贴板。参见 § 8.30。
粘贴	Ctrl+V	将剪贴板内容插入到所选内容旁：步骤插入到所选步骤之后，组插入到所选步骤中，选项插入到所选组中。参见 § 8.30。

3.3 项目菜单

所有作用于正在编辑的 FOMOD 的操作。

命令	快捷键	描述
验证 FOMOD	—	检查项目以及 ModuleConfig.xml 对 ModConfig 5.0 架构的符合性，核查所引用的文件、目标路径冲突和图像（在后台进行），然后显示一份详细报告。参见 § 4.8。
预览安装程序…	—	按模组管理器显示的方式模拟安装程序。参见 § 4.11。
查找缺失文件…	—	列出项目使用但已不存在的文件、文件夹和图像，并提示它们的去向。参见 § 8.32。
条件编辑器…	—	列出每个条件标志及其设置和使用位置（可点击），可在各处重命名或移除某个标志，并为所选节点提供一个条件构建器。参见 § 8.7 和 § 8.22。
翻译 FOMOD…	—	打开一个窗口，用于翻译现有安装程序（活动项目、一个文件夹或一个存档），而不改动其 XML 结构。参见 § 8.13。

命令	快捷键	描述
与…比较	—	将项目与另一个 FOMOD 进行比较，并列出有何变化。参见 § 8.6。
模板…	—	将所选步骤保存为可复用模板，或插入一个已保存的模板。参见 § 8.3。
预设向导…	—	在当前项目上生成一个“预设 + 自定义”安装程序：一个包含各预设的安装模式页面、按需显示的功能页面、动态默认值以及条件安装。参见 § 7.5 和 § 8.29。
XML 编辑器	—	提供访问 info.xml 和 ModuleConfig.xml 的子菜单。参见 § 4.10。

3.4 工具菜单

与特定项目无关的设置和实用工具。

命令	快捷键	描述
设置…	Ctrl+,	打开“设置”窗口（国家/地区、语言、主题、字体等）。参见 § 4.5。
树颜色…	—	为项目树中的每种行（步骤、组、选项…）分别选择一种颜色，每个主题各一套。参见 § 8.27。
保存前脚本…	—	编辑保存前运行的脚本。参见 § 4.6。
保存后脚本…	—	编辑保存后运行的脚本。参见 § 4.6。
插件报告…	—	列出项目所安装的每个插件文件（.esp、.esm、.esl）的表格：类型、light 标志、主文件（缺失的以高亮显示）、相对于轻量插件上限的新建记录数以及 ESL 资格。参见 § 8.18。
压缩包内容…	—	列出打包在 Bethesda .bsa 或 .ba2 存档中的文件（格式、大小、压缩情况），并带有搜索字段。参见 § 8.24。
Nexus 描述…	—	根据项目生成 Nexus Mods 页面文本（BBCode 或 Markdown）：依赖项、安装选项、安装说明，以及相对于先前版本的更新日志；可复制或保存。参见 § 8.21。
项目字符串…	—	列出项目的每个名称和描述的表格，带有搜索、查找/替换和重复项检测；文本可就地编辑。参见 § 8.23。
翻译界面…	—	打开界面翻译编辑器。参见 § 4.9。
属性…	—	打开国家/地区 / 语言数据库的只读浏览器。参见 § 4.14。

命令	快捷键	描述
游戏 / 分类…	—	打开游戏编辑器：无需手动编辑 Categories.json，即可添加、修正或删除游戏及其 Nexus 分类（名称、slug、基础主文件、轻量插件上限）。参见 § 8.26。

3.5 帮助菜单

命令	快捷键	描述
用户手册	F1	从随应用附带的 Manuals 文件夹中，以界面语言打开本手册（PDF）。
检查更新…	—	查询最新发布的版本并提供下载链接。
关于	—	显示“关于”窗口（名称、版本、许可证）。参见 § 4.7。

4. 功能详解

4.1 模组信息标签页

此标签页汇集了模组元数据以及根目录的选择。

工作区 —— “根目录：”一行显示工作文件夹（若未定义则显示“（请选择一个目录）”）。“浏览...”按钮用于选择它。

各元数据字段如下：

字段	作用
模组名称	安装程序的显示名称（必填）。
作者	模组作者的名字。
版本	版本号（新项目默认为 1.0.0）。
游戏名称	目标游戏，从取自 Categories.json 的列表中选择。
分类	模组的分类。取决于所选游戏；否则会提供一个默认列表（动画、护甲、音频、纹理、武器等）。
网站网址	模组页面的地址。
封面图片	显示在安装程序顶部的图像。有“浏览...”和“清除”按钮；会显示预览。路径相对于根目录存储。
描述	模组的介绍文本（多行区域）。

> 注意 —— 对任何字段的更改都会在状态栏中将项目标记为“已修改”。

Nexus 链接 —— 在“游戏名称”选择器旁，有一个“Nexus ↗”按钮，可直接在浏览器中打开所选游戏的 Nexus Mods 页面（根据其 “slug” 构建）。仅当所选游戏具有 slug 时该按钮才有效。

“游戏名称”和“分类”列表来自外部文件 assets/data/Categories.json，无需重新编译即可编辑。每款游戏在其中由一个标识符、一个名称、一个 Nexus “slug” 及其分类列表来描述：

```
{
  "games": {
    "skyrimSpecialEdition": {
      "name": "The Elder Scrolls V: Skyrim Special Edition",
      "nexusSlug": "skyrimspecialedition",
      "categories": ["Animation", "Armour", "Audio", "…"]
    }
  }
}
```

4.2 安装步骤标签页

这是安装向导的核心。它基于一个三级层次结构：步骤 → 组 → 选项，这正是左侧项目树的形态。用“安装步骤”旁的 + 按钮（默认命名为“步骤 N”）或从空检查器中添加一个步骤；在树中选中一个步骤、组或选项即可在中央编辑它。步骤编辑器显示名称（可编辑）、“删除步骤”按钮（唯一会要求确认的删除操作：其余一切都可用

Ctrl+Z
撤销）、可见性条件、页面目标以及带有“添加组”按钮的组列表；组编辑器显示名称、选择类型、组目标以及带有“添加选项”按钮的选项列表；选项编辑器将在下文说明。

组与选择类型

每个组都有一个名称和一个选择类型，用以确定最终用户可勾选的选项数量。可用的五种类型是：

类型	含义
恰好一个（必选）—— SelectExactlyOne	用户必须恰好选择一个选项。
至多一个 —— SelectAtMostOne	至多一个选项（或一个都不选）。
任意数量 —— SelectAny	任意数量的选项（默认值）。
全部（无选择）—— SelectAll	所有选项都被选中。
至少一个 —— SelectAtLeastOne	至少一个选项。

步骤编辑器的“添加组”按钮（或步骤的右键菜单）会创建一个组，默认类型为“任意数量”；组的右键菜单或组编辑器中的“删除”会将其移除。

重新排序 ——
显示顺序的调整无需重建一切：在树中拖动一个节点并放到它应在的位置（一个选项放到另一个选项之前或放到一个组上、一个组放到另一个组之前或放到一个步骤上、一个步骤放到另一个步骤之前），或使用节点右键菜单中的“移至前面” /
“移至后面”。“创建副本”会连同所含内容一并复制一个步骤、组或选项。每项更改都可用 编辑 →
撤销（Ctrl+Z）撤销，并可用 编辑 →
重做（Ctrl+Y）重做；字段中的连续按键算作一步。剪切、复制和粘贴——
位于同一右键菜单、编辑菜单中，或使用 Ctrl+X / Ctrl+C / Ctrl+V——
可将节点移到项目中的其他位置，或移入另一个已打开的 FOMOD（§ 8.30）。

选项与默认类型

一个选项（FOMOD 格式中的 <plugin>
元素）是提供给用户的一个选择。它具有名称、描述、图像（预览、“浏览…”和“清除”按钮）以及一个默认类型：

类型	含义
可选 —— Optional	可选的选项（默认）。
必需 —— Required	始终安装的选项。
推荐 —— Recommended	推荐但可选的选项。
不可用 —— NotUsable	不可用的选项。
可能可用 —— CouldBeUsable	视条件而定可能可用的选项。

插件文件

“源文件”部分以四列表格（类型、源文件、目标文件、优先级）呈现该选项所安装的文件和文件夹。“添加文件”、“添加文件夹”和“移除”按钮用于管理该列表。“目标文件”列可编辑：单击某个

单元格并输入相对于游戏 Data 文件夹的路径，用于指定该项必须安装的位置。该列默认为空，表示将文件——或文件夹的内容——安装到 Data 的根目录；当源文件夹本身已重现游戏的目录结构时（例如一个本身包含 meshes、textures、SKSE 等的文件夹），这是正确的选择。仅当目标不同时才输入明确路径，例如 SKSE\Plugins\MyMod.ini。文件必须位于根目录之下。在选中该选项时，您也可以直接从文件资源管理器（Windows 资源管理器、Finder…）将文件或文件夹拖放到 XIMOD 窗口上：它们会立即被添加到该选项的“源文件”表格中（参见 § 8.5）。对话框会在您上一次所选的文件夹中打开，因此同级文件夹只需点击一下即可到达（§ 8.28）。“在磁盘上重命名…”会在模组文件夹中重命名所选文件或文件夹，并更新对它的所有引用（§ 8.31）。

组 / 页面目标

若要将多个选项发送到同一位置而无需逐一编辑每个文件，可使用批量目标控件。在某个组的编辑器中，“整个组的目标”字段及其“应用到此组中的所有选项”按钮可一次性为该组中每个选项的每个文件设置同一个安装目标。在步骤层级，可折叠的“安装目标（整个页面）”面板对页面上横跨所有组的每个选项执行相同操作。一条状态消息会确认更新了多少个文件。在选项实际包含文件之前该按钮始终禁用，并且之后您仍可在选项的“源文件”表格中微调任何单独的目标路径。这些控件同样作用于预设向导（§ 8.29）移到条件安装中的文件：由该组或页面的选项所填充的条件集也会一并更新。

条件标志

“标志名称：”部分用于将标志与选项关联（名称 = 值对）。当最终用户选择某个选项时，会设置其标志；随后这些标志可用于约束步骤的可见性、选项的类型（依赖模式）或条件文件的安装。“添加标志”和“移除标志”按钮用于管理该列表。

自动补全 —— 标志和依赖项的名称与值字段会在您输入时提供一个建议菜单，其内容取自项目中已有的元素，从而避免因拼写错误而悄然破坏匹配。对于文件依赖，值字段会建议 Active / Inactive / Missing 这几种状态。这些建议并非强制：仍可自由输入新值。

选项依赖项。在标志下方，此部分使选项的类型变为动态。上方的“默认类型”在无任何模式匹配时适用；每个模式会在其条件满足时赋予该选项另一种类型。使用“新建模式” / “删除模式”管理该列表；为所选模式选择“模式运算符”（And / Or）和要应用的“模式类型”（Optional、Required、Recommended、NotUsable 或 CouldBeUsable），然后添加其条件。一个条件具有类型（File 或 Flag）、一个 File/Flag 名称（带自动补全），以及——对文件而言——一个 State（Active、Inactive 或 Missing），或者——对标志而言——其期望值。例如，将“默认类型”设为 NotUsable，并添加一个 Optional 类型的模式，其条件为文件 DLCCoast.esm 处于 Active 状态，这样该选项仅在安装了 Far Harbor 时才可选。

步骤可见性条件。每个步骤都可有条件地显示。在步骤名称下方，可折叠的“可见性条件”部分保存着该步骤的可见性规则：设置“运算符”（And / Or）并以相同方式添加 File 或 Flag 条件。当它包含至少一个条件时，该步骤仅在条件满足时才在向导中出现——例如一个仅在 DLCCoast.esm 处于 Active 时才显示的补丁步骤。保持它为空则始终显示该步骤。

4.3 必需安装项标签页

此标签页列出无论用户如何选择都会系统性安装的文件和文件夹。它呈现与选项文件部分相同的四列表格（类型、源文件、目标文件、优先级）——带有同样可编辑的“目标文件”列——以及相同的“添加文件”、“添加文件夹”和“移除”按钮。表格上方的“目标文件”字段及其“应用于所有必需文件”按钮可一次性为每个必需文件设置同一个安装目标——相当于 § 4.2 中的组 / 页面目标控件；之后仍可微调每个目标。

4.4 条件安装项标签页

此标签页将各模式分组：每个模式仅在其条件满足时才安装一组文件。一行“模式 N”标签让您可添加（“＋”）或移除（“－”）一个模式。

对于所选模式：

- 运算符 —— And 或 Or：该模式的各依赖项如何组合。
- 依赖项 —— 一个由 Flag（[Flag] 名称 = 值）或 File（[File] 名称 (状态)）类型的条件组成的列表。“添加依赖”和“移除依赖”按钮用于管理该列表。
- 文件 —— 条件满足时所安装文件的四列表格。其上方的“目标文件”字段及其“应用于此集中的所有文件”按钮可一次性为该模式中的每个文件设置同一个安装目标——在预设向导（§ 8.29）将选项的文件移到此处之后尤为方便。在标签页顶部，“目标文件”字段及其“应用于所有条件集”按钮可一次性为每个集中的每个文件设置同一个安装目标——使用预设向导之后，每个选项可能各有一个集。

注意 ——
“插件依赖”。一个插件依赖其实就是一个文件依赖，只不过其文件是一个游戏插件（.esm/.esp）。在名称字段中，输入该插件在游戏中显示的文件名（例如 Fallout4.esm、DLCCoast.esm，或另一个模组的 .esp），而非模组的显示名称。选择状态：Active（插件存在且已启用——通常的选择）、Inactive（存在但已禁用）或 Missing（不存在）。文件依赖可用于三处：自动安装文件（“条件安装项”标签页）、显示或隐藏某个向导步骤（步骤的 visible 条件），或更改某个选项的状态——例如当某个 DLC 处于活动状态时使其成为 Recommended，或当缺少某个必需插件时使其成为 NotUsable。

4.5 “设置”窗口

通过 工具 → 设置（Ctrl+,）访问。它有两个标签页：“常规”和“最近文件”。设置保存在 Config.ini 中。

“常规”标签页分为两栏。

左栏 —— 国家/地区旗帜及其名称：

项目	作用
旗帜	单击该图像会打开国家/地区选择器（见下文）。
国家/地区名称	以所选语言书写的国家/地区名称（本族名）。不可编辑字段，分两行显示。

右栏 —— 界面设置：

设置	描述
语言	界面语言，立即生效。只要未选择国家/地区，此字段便处于禁用状态；选择后它会提供该国家/地区所使用的语言。
主题	深色、浅色或跟随系统。
字体大小	界面文本的大小。
处理换行	解释描述中的 \n 序列。
最大最近文件数	保留的条目数量。
窗口宽度 / 高度	主窗口的已存尺寸。

国家/地区选择器 —— 单击旗帜会打开“国家/地区选择”窗口，它以网格形式呈现 244 面旗帜，可垂直和水平滚动。它是一个自由窗口，打开时居中于主窗口（参见 § 2.6）。“筛选：”字段让您可按法文名称、英文名称或国家/地区代码缩小列表；字段右侧的清除图标（红色叉号）单击即可清空筛选。每面旗帜下方显示国家/地区名称。

该网格也可用键盘导航：方向键（或数字小键盘 8/2/4/6）移动选择框，Page Up/Page Down 滚动一屏，Home/End 跳到第一面和最后一面旗帜，Tab 和 Shift+Tab 向前和向后移动一面旗帜，Enter 确认高亮的国家/地区，Esc 关闭窗口。详情见附录 B。

国家/地区的选择决定了所提供的语言列表：XIMOD 依赖 CountryLanguages.json 文件，该文件列出了每个国家/地区所使用的所有语言——巴布亚新几内亚多达 844 种，这正是下拉列表需要滚动的原因。

版本 2 新增了三项设置：“保留的备份数”（每次保存时保留在 fomod/backups 下的 FOMOD XML 历史版本数量，0 = 不保留，默认 10）；“自动保存恢复副本的间隔（分钟）”（按此间隔将每个已修改项目的恢复副本写入配置文件夹，仅在异常退出后于下次启动时提供，0 = 关闭，默认 5）；以及“将插件的主文件添加为条件”（参见 § 8.17，默认开启）。

4.6 保存前脚本与保存后脚本

XIMOD Architect 可在保存时自动运行脚本：保存前脚本在 FOMOD 文件写入之前启动，保存后脚本则在之后启动。它们用于自动化模组生成前后的任务——例如在保存前清理旧存档、递增构建号，或在 FOMOD 写入后自动将模组打包为可直接发布的存档。

它们通过 工具 → 保存前脚本… 和 工具 → 保存后脚本… 来编辑。该窗口显示一段帮助文本、可用宏的列表、一个等宽字体的输入区，以及“保存”和“取消”按钮。两个脚本的内容都保存在 Config.ini 中（键 PreSaveScript 和 PostSaveScript），因此会在各次会话之间被记住。

工作原理：在保存时，XIMOD 首先将宏替换为其值，将结果写入一个临时文件，然后用系统解释器运行它——在 Windows 上由 cmd 启动的 .bat 命令文件，在 Linux 和 macOS 上由 sh 启动的 .sh shell 脚本。临时文件随后被删除。保存前脚本在 XML 文件写入之前运行，保存后脚本则在之后立即运行；留空的脚本会被直接忽略。

执行前，脚本文本中会替换以下宏：

宏	替换后的值
$MODNAME$	模组名称。
$MODAUTHOR$	作者名字。
$MODVERSION$	模组版本。
$MODROOT$	根目录的路径。
$DATE$	当前日期（YYYY-MM-DD）。
$TIME$	当前时间（HH:MM:SS）。
$RANDOM$	随机数。

示例 —— 保存前脚本（Windows）

删除已存在于模组根目录下的 .rar 存档，以便在保存前从一个干净的文件夹开始：

```
del "$MODROOT$\*.rar" /q
```

示例 —— 保存后脚本（Windows）

FOMOD 写入后，使用 WinRAR 自动将整个模组内容打包为一个以模组名称和版本命名的 .rar 存档：

```
d:
cd "$MODROOT$"
"C:\Program Files\WinRAR\rar.exe" a -r -ep1 "$MODNAME$_$MODVERSION$.rar" *
```

逐行说明：d: 切换到 D: 盘，cd "$MODROOT$" 移动到模组的根目录，然后 WinRAR 创建存档（-r 递归、-ep1 以免重现上级目录树），命名为例如 Dawnlight_1.2.0.rar。

Linux / macOS 等效写法

在 Linux 和 macOS 上，脚本被解释为 shell 脚本。同样的保存后打包操作，例如用 zip 编写如下：

```
cd "$MODROOT$"
zip -r "$MODNAME$_$MODVERSION$.zip" .
```

> 注意 —— 这些脚本以您的用户权限运行真实的系统命令；像 del 或 rm
> 这样的命令具有破坏性。请先在一个可随意丢弃的项目上测试它们，然后再用于真实的模组。

4.7 “关于”窗口

通过 帮助 → 关于 打开，此窗口展示应用名称、其版本号、一段简短描述（“一款用于 Vortex 或 Mod Organizer 2 能够管理的任何游戏模组的跨平台 FOMOD 安装程序生成工具。”）以及说明“Licensed under MIT”。一个“确定”按钮将其关闭。

4.8 生成 FOMOD 文件

保存时，XIMOD Architect 会将项目转换为两个标准化的 XML 文件，以 UTF-8 写入 fomod 子文件夹。

写入之前，会对项目进行检查：如果它包含异常（缺少模组名称、没有步骤或必装文件、某个步骤或组没有名称、某个组没有选项），会有一个窗口将它们全部列出并提供“仍然保存”——保存从不会被阻止，这让您可以有意地保存一个进行中的项目。

架构验证 —— 在此项目检查之外，还会对所生成的 ModuleConfig.xml 进行针对 ModConfig 5.0 架构的结构性检查，该检查用 Rust 编写（无需外部 XSD 引擎）：元素层次与基数、强制属性和枚举值（选择类型与插件类型、文件状态、运算符）。每个异常都会按其行和列定位。它可通过三种方式访问：项目 → 验证 FOMOD 命令，显示完整报告；在 XML 编辑器中，一旦文档格式良好便实时显示；以及集成在保存确认窗口中。

info.xml 包含元数据，置于 <fomod> 根下：名称（Name）、作者（Author）、版本（Version）、网站（Website）、描述（Description）、分类（Groups）和游戏（Game）。Game 元素是 XIMOD 特有的扩展，会被模组管理器无害地忽略，用于在重新加载时恢复正确的分类列表。

ModuleConfig.xml 描述安装过程，置于 <config> 根下：模块名称、页眉图像、必装文件（requiredInstallFiles）、步骤（installSteps → group → plugin），以及条件安装（conditionalFileInstalls）。每个 plugin 都描述其描述、图像、条件标志、文件和类型。依赖项由一个 And 或 Or 运算符组合。

4.9 翻译编辑器

通过 工具 → 翻译界面... 访问，此工具让您无需手动编辑文件即可翻译界面，并修正现有翻译。

窗口页眉

字段	作用
旗帜	国家/地区的选择，它决定了可翻译语言的列表。
国家/地区本族名	以所翻译语言显示的国家/地区名称，并以所选字体显示。
语言本族名	该语言用其自身书写的名称（例如“Français”）。
作者	翻译者的名字。
显示语言	在第 2 列显示的参考语言（取自已有 main.ftl 文件的那些语言）。
要翻译的语言	目标语言，从该国家/地区的语言中选择。
字体	用于该语言的字体。“浏览...”按钮会打开一个定位在 assets/fonts 的文件选择器。
Google 字体	在浏览器中打开 fonts.google.com 网站。

重要约束：字体必须安装在 assets/fonts 文件夹中。位于其他位置的字体会被拒绝，因为程序通过相对于该文件夹的路径加载它，否则在下次启动时将无法找到它。因此，如果缺少该字体，您需要先从 Google Fonts 下载它并复制到那里，然后再选择它。

同样地，只有具有 ISO 639-3 代码的语言才能被翻译：正是这个代码为翻译文件夹命名，并将语言与其字体和参考数据相关联。否则保存按钮会被禁用。

翻译表格 —— 三列：

第 1 列	第 2 列	第 3 列
键（不可编辑）	以显示语言呈现的标签	翻译输入字段

标记保护：不应被翻译的元素（诸如 $MODNAME$ 的宏、诸如 { $num } 的变量）绝不会在第 2 列中按原样出现，并会被自动保留。

- 位于文本开头或结尾的标记：会被隐藏，然后在保存时重新插入。
- 位于文本中间的标记：会被替换为紧凑标记 [[1]]、[[2]]…以便保留并重新定位。

应用名称（键 app-title）被特意排除，不可翻译。

键盘导航 —— 该表格可不用鼠标操作（参见附录 B）：上/下方向键和 Page Up/Page Down 滚动各行，Home/End 跳到第一行和最后一行，Enter 将光标置于高亮行的输入字段中。进入编辑后，Tab 移至下一字段，Shift+Tab 移至上一字段，从而可以一个接一个地输入翻译而无需离开键盘。Page Up/Page Down 移动的行数会自动适应窗口高度。

保存 —— “保存”按钮会写入文件 assets/locales/<语言>/main.ftl，前面附有一个以 Fluent 注释书写的元数据块（因此该文件仍然完全有效）：

```
# XIMOD Architect - translation metadata
# @language = fra
# @font = Noto_Sans/static/NotoSans-Regular.ttf
# @langname = Français
# @author = Patrice
```

页眉仅描述语言：它不包含任何国家/地区信息，因为同一种语言会在多个国家/地区使用（德语在德国使用，也在瑞士使用）。国家/地区由所选旗帜确定，XIMOD 根据 Countries.json 和 CountryLanguages.json 处理 国家/地区 → 语言 的映射。

填写本族名字段 —— 当国家/地区（旗帜）和语言选定后，XIMOD 会填写这两个字段：

- 国家/地区本族名：始终根据所选国家/地区从 Countries.json 解析得出；因此它随旗帜而变（对德国，德语显示“Deutschland”，对瑞士显示“Schweiz”）。
- 语言本族名：当文件页眉（# @langname）中有该信息时从中获取；若无——一个没有页眉的旧文件，或一种尚未翻译的语言——则从 Languages.json 获取。

这两个字段仍可编辑：用户可在此处进行修正，这些修正会在保存时传播到参考文件。

保存会更新参考数据：Languages.json 中该语言的本族名和字体，以及 Countries.json 中的国家/地区本族名（针对该 国家/地区 / 语言 对）。如果该语言在此国家/地区下尚未出现，则会创建一个条目。

由于页眉不再携带国家/地区信息，XIMOD 会在 main.ftl 旁边写入一个以制表符分隔的小文件 countryEndonyms.tsv，它记录该语言下的国家/地区本族名，每个国家/地区一行，形式为“国家/地区代码，语言代码，本族名”。正是这个文件使得仅有语言文件夹可用时也能恢复国家/地区本族名。

发送翻译 —— “发送…”按钮会构建一个 ZIP 存档，其中包含翻译文件（main.ftl，位于其语言文件夹中），以及（如果存在）随附的 countryEndonyms.tsv 文件；XIMOD 会打开存档所在文件夹，然后准备一封预填好的电子邮件。语言专有信息（语言、字体、语言本族名、作者）包含在 main.ftl 的页眉中；国家/地区本族名包含在 countryEndonyms.tsv 中。Languages.json 和 Countries.json 均不会被附上——仅传送实际使用到的代码，这尊重了 ISO

639-3
代码的使用条款。用户自行附加文件并自行发送：未经明确操作，任何数据都不会离开本机。

> 注意 ——
> 仅在翻译完成后才发送该包：任何未翻译的键都会以空值保存，随后会在界面中无文本显示。

4.10 XML 编辑器

通过 项目 → XML 编辑器 → info.xml 或 ModuleConfig.xml 访问，此面向高级用户的工具让您可查看并直接修改所生成的 XML。

它在一个独立窗口中打开，可自由移动和调整大小，包括在第二块屏幕上。

两种模式

- *只读（默认）：XML 以语法高亮和悬挂缩进显示——*
  过长的行会换行，对齐在行首之下，方式类似 Notepad++。主窗口仍可使用。
- *编辑（“修改”按钮）：文本变为可编辑，主窗口的各标签页被锁定，以确保单一来源具有权威性。*

语法高亮：标签、属性名、带引号的值、注释、声明和文本会以适合浅色或深色主题的颜色加以区分。

行号：侧栏显示行号；换行的续行保持空白。

实时验证：在输入时，XML
会被持续检查。绿色指示器确认文档格式良好；若出错，一条红色消息会指明行和列，并将出错的标记下划线标出。

应用更改：“应用”按钮会重新解析文本并更新项目。若出错，更改会被拒绝，文本保持可编辑状态。“取消”按钮会从项目重新生成文本。

4.11 字体管理

XIMOD Architect 附带一套 Noto 字体，涵盖所列语言的所有书写系统，置于 assets/fonts 中。

这些字体会被动态加载：任何时候只有实际需要的字体才会被安装，即界面语言的字体以及所选国家/地区各语言的字体。负担保持很低——对大多数国家/地区而言只需一种字体。

此机制可确保非拉丁文字（泰文、缅甸文、阿姆哈拉文、苗文等）的正确显示，否则它们会显示为空方块。它同时适用于界面以及“属性”窗口和翻译编辑器：在那里显示的每种语言的字体都会被加载，使任何名称或标签都不会显示为方块。

XIMOD Architect 附带 32 种界面翻译，每种都涵盖全套字符串，包括非拉丁文字：日语（jpn）、简体中文（zho）、韩语（kor）和俄语（rus）。英语（eng）和法语（fra）是参考翻译，由人工维护；其余的来自贡献，母语使用者可通过翻译编辑器进行审校。

每种语言所关联的字体在 Languages.json 中指明，并可从翻译编辑器中更改。

4.12 合并两个 FOMOD

文件 → 合并 FOMOD… 命令让您通过合并两个安装程序的安装内容来组合它们。当前打开的项目充当接收方；因此它必须已事先创建或打开，否则该命令保持禁用。

单击会打开一个文件选择器：选择要整合的 FOMOD（供给方）的 ModuleConfig.xml 文件。随后它的步骤、必装文件和条件安装会被添加到接收方项目相应内容的末尾。接收方项目的元数据——名称、作者、版本、分类、页眉图像——会被保留：仅导入安装内容。

合并后，项目会被标记为已修改：请记得保存它（Ctrl+S）以将结果写入 FOMOD 文件。可重复进行合并，以便一个接一个地整合多个 FOMOD。

> 注意 —— 仅读取供给方的 ModuleConfig.xml 文件；其自身的元数据（info.xml）不会被导入。合并后，请检查在组合后的各步骤中优先级和条件是否仍然一致。

4.13 导出分发存档

文件 → 导出分发存档… 命令可通过单次操作生成可直接上传（Nexus Mods 或任何其他模组管理器）的存档。XIMOD 首先将 FOMOD（fomod/info.xml 和 fomod/ModuleConfig.xml）写入根目录，然后以 ZIP 格式压缩整个文件夹。只要未定义根目录，该命令便禁用。

一个选择器会提供形如“<名称>-<版本>.zip”的默认名称（已清除禁用字符）；您可在保存前更改它。随后状态栏会指出所写入文件的数量以及存档的路径。

存档布局 —— fomod/ 子文件夹和模组文件位于存档的根部，与模组管理器所期望的完全一致。内部路径使用正斜杠“/”（跨平台兼容性）。

> 注意 —— 垃圾文件（.git、.DS_Store、Thumbs.db、desktop.ini…）以及存档本身会被自动排除。因此，请确保在导出前所有模组文件确实位于根目录之下。

4.14 “属性”窗口

通过 工具 → 属性… 打开，此独立窗口——可移动和调整大小，包括在第二块屏幕上——提供内嵌的 国家/地区 / 语言 数据库的只读浏览器。此处无法修改该数据库；它是一个查阅工具。

“国家”标签页：对于所选的国家/地区，该窗口显示其旗帜、其法文和英文名称、其官方语言（带本族名、字体和 ISO 639-3 / 639-1 代码）以及在那里所使用的所有语言的列表。

“语言”标签页：对于所选的语言，该窗口显示其 ISO 代码、其字体，并执行反向查找，列出使用此语言的国家/地区。“使用语言”列表以您在“设置”中所选的语言（旗帜 → 国家/地区 → 语言）命名每个国家/地区：当该语言在当地为官方语言时，使用 Countries.json 中该语言下的本族名（对意大利语用户为“Confederazione Svizzera”），否则使用 CountryNames.json 中的译名（对德语用户为“Südafrika”），并以英文名称作为最后手段。

键盘导航 —— 国家/地区列表和语言列表均可用键盘操作（参见附录 B）：上/下方向键、Page Up/Page Down（其步长会适应窗口高度）、Home/End 和 Enter（选中高亮的项）。Tab 和 Shift+Tab
在列表内移动选择，永远不会落到旗帜上。每个“筛选：”字段的右侧都有一个清除图标（红色叉号），窗口以 Esc 关闭。

4.15 命令行模式（CLI）

以参数启动时，XIMOD Architect
不打开任何窗口：它执行所请求的命令，然后以一个退出码终止（成功为
0，失败或验证异常为非零）。此模式使得可以在脚本或持续集成流水线中自动化地重建和打包一个或多个 FOMOD。

> 注意 —— 在 Windows
> 上，由于可执行文件被编译为“windows”应用程序，它会附加到启动它的控制台（通过 AttachConsole）以在那里写入其消息。请从命令提示符（cmd 或 PowerShell）启动它以查看输出；双击一条 CLI 命令将不会显示任何内容。

可用的命令如下：

命令	效果
ximod-architect validate <root>	验证该目录的 FOMOD（项目检查以及对 ModConfig 5.0 架构的符合性）；每个异常都会被定位。若发现异常则退出码为非零。
ximod-architect build <root>	从现有项目（重新）写入 fomod/info.xml 和 fomod/ModuleConfig.xml。
ximod-architect package <root> [-o FILE]	写入 FOMOD，然后构建 .zip 分发存档。-o 选项让您指定输出路径；若不指定，存档将在当前目录中创建。
ximod-architect help	显示帮助（命令列表）。
ximod-architect version	显示版本号。

此处，<root> 表示模组的根目录（包含——或将要包含—— fomod 子文件夹的那个文件夹）。

4.16 检查更新

默认情况下，XIMOD Architect
会在启动时每天检查一次是否有更新版本可用。如果有，菜单栏下方会出现一个不显眼的横幅，带有下载它的链接（Nexus 和 GitHub）；您也可随时从 帮助 → 检查更新
进行检查。这可在“设置”窗口中关闭（“启动时检查更新”）。
隐私：此检查是本应用发出的唯一网络请求。它不发送任何个人数据——仅向公开的 GitHub releases 页面发出单次安全（HTTPS）请求以读取最新版本号。XIMOD
绝不会自行下载或安装任何内容；是否点击下载链接始终由您选择。

5. FOMOD 标签参考

本节描述 XIMOD Architect 实际生成的标签。两个文件都以 UTF-8（带字节顺序标记）写入模组的 fomod 子文件夹：info.xml 以四空格步进缩进，ModuleConfig.xml 以制表符缩进。读取时，编码会借助字节顺序标记自动检测，从而可以打开由其他工具生成的 UTF-8 和 UTF-16（LE 或 BE）文件。

5.1 info.xml 文件

元数据，置于 <fomod> 根下。各元素按下表顺序写入；标注“非空时”的元素在相应字段留空时会被省略。

标签	内容	写入
<Name>	模组名称。	始终
<Author>	作者。	非空时
<Version>	版本。	非空时
<Website>	模组页面的 URL。	非空时
<Description>	描述。	非空时
<Groups>	模组的分类。	始终
<Game>	所选游戏的标识符。XIMOD 特有的扩展（在 FOMOD 规范之外），会被模组管理器忽略。	非空时

完整示例：

```
<?xml version="1.0" encoding="utf-8"?>
<fomod>
    <Name>My Awesome Mod</Name>
    <Author>Patrice</Author>
    <Version>1.0.0</Version>
    <Website>https://www.nexusmods.com/skyrimspecialedition/mods/12345</Website>
    <Description>A sample mod for the manual.</Description>
    <Groups>Player homes</Groups>
    <Game>skyrimSpecialEdition</Game>
</fomod>
```

5.2 ModuleConfig.xml 文件

安装逻辑，置于一个引用 ModConfig5.0.xsd 架构的 <config> 根下。各块按以下顺序出现：模块名称、页眉图像、必装文件、安装步骤、条件安装。

根与页眉

标签 / 属性	作用
<config>	根。携带 xmlns:xsi 和 xsi:noNamespaceSchemaLocation 属性（ModConfig5.0.xsd 架构）。
<moduleName>	模块名称，显示在向导顶部。
<moduleImage path="…"/>	页眉图像（空标签，仅在定义了图像时才写入）。

文件与文件夹

标签 / 属性	作用

标签 / 属性	作用
`<file source="…" destination="…" priority="…"/>`	复制一个文件。source = 存档中的路径（相对于根）；destination = Data 下的目标；priority = 安装顺序。
`<folder source="…" destination="…" priority="…"/>`	复制整个文件夹（属性相同）。
`<requiredInstallFiles>`	系统性安装的文件的容器。
`<files>`	与某个选项或条件模式关联的文件的容器。

步骤与组

标签 / 属性	作用
`<installSteps order="Explicit">`	各步骤的容器（所定义的顺序会被保留）。
`<installStep name="…">`	向导的一个页面。
`<visible>`	步骤的可见性条件（包含 <dependencies>）。
`<optionalFileGroups>`	步骤中各选项组的容器。
`<group name="…" type="…">`	一个组；type = 选择规则（见下文）。
`<plugins order="Explicit">`	组中各选项的容器。

<group> 的 type 属性的取值：

type	约束
恰好一个（必选）—— SelectExactlyOne	恰好一个选项。
至多一个 —— SelectAtMostOne	至多一个选项。
任意数量 —— SelectAny	任意数量。
全部（无选择）—— SelectAll	全部，强制。
至少一个 —— SelectAtLeastOne	至少一个。

选项（plugins）

标签 / 属性	作用
`<plugin name="…">`	提供给用户的一个选项。
`<description>`	选项的描述文本（始终写入）。
`<image path="…"/>`	说明该选项的图像（如已定义）。
`<conditionFlags>`	选中该选项时所设置的标志的容器。
`<flag name="…">value</flag>`	一个标志：名称在属性中，值在内容中。
`<typeDescriptor>`	描述选项的类型（简单型或条件型）。
`<type name="…"/>`	简单类型（当未定义依赖模式时）。
`<dependencyType>`	动态类型（当存在模式时）。
`<defaultType name="…"/>`	当无任何模式适用时所应用的默认类型。

<type> / <defaultType> 的取值：

类型	在向导中的效果
可选 —— Optional	选项未勾选，可自由激活。
必需 —— Required	强制性选项，无法取消选择。
推荐 —— Recommended	默认勾选的选项，可取消选择。
不可用 —— NotUsable	被禁用的选项（无法选择）。
可能可用 —— CouldBeUsable	可用，但标记为可能有风险。

依赖与条件

标签 / 属性	作用
`<dependencies operator="…">`	条件组；operator = And 或 Or。
`<fileDependency file="…" state="…"/>`	对某个文件的条件；state = Active、Inactive 或 Missing。
`<flagDependency flag="…" value="…"/>`	对某个标志及其期望值的条件。
`<conditionalFileInstalls>`	条件安装部分。
`<patterns> / <pattern>`	模式列表；每个模式将 <dependencies> 与 <files> 关联（或在 typeDescriptor 内与一个 <type> 关联）。

6. ModuleConfig.xml 文件详解

第 5 章以参考表格的形式介绍了各标签。本章会逐一讲解它们，用通俗语言解释每个标签的作用、它接受哪些选项，以及最重要的——
这些选项对模组用户将看到的安装向导有何具体影响。无需任何 XML 先验知识：XIMOD Architect 会为您编写此文件；理解它只是有助于您设计更可靠的安装程序。

几个术语说明。一个 XML 文件由尖括号中的“标签”组成。一个开始标签 <step> 与一个结束标签 </step> 配对；两者之间写的内容都属于该标签。一个标签可携带“属性”，即以 名称="值" 形式书写的设置。于是 <group name="Resolution" type="SelectExactlyOne"> 就是一个带有两个属性的“group”标签。最后，没有内容的标签会自我闭合，如 <file ... />（注意结尾的 /）。

6.1 安装程序如何读取文件

当用户安装您的模组时，他们的管理器（Vortex、MO2、NMM）会自上而下地读取 ModuleConfig.xml
并构建一个向导：一系列页面（步骤），每个页面提供供勾选的选项组。最后，管理器会收集与所选选项相匹配的所有文件并将它们复制到游戏中。需要记住三个要点：

- 顺序很重要：步骤按其在文件中出现的顺序显示；
- 勾选不会立即安装任何内容：勾选某个选项只是记录一个选择；文件的实际复制发生在最后；
- 条件可以调节一切：隐藏某个步骤、更改某个选项的状态或触发额外文件的安装——这就是标志和依赖的作用（§ 6.6）。

6.2 根与页眉

<config> —— 根。整个配置都位于单个 <config>…</config> 标签内。它携带两个技术属性（xmlns:xsi 和 xsi:noNamespaceSchemaLocation），用以指定验证架构（ModConfig 5.0）。您永远无需触碰它们：XIMOD 始终以相同方式写入它们。

<moduleName> —— 模组名称，显示在向导顶部。它就是在“模组信息”标签页中输入的那个。

<moduleImage path="..."/> —— 显示在向导顶部的横幅图像。path 属性指明其位置，相对于模组的根（例如 fomod\header.png）。该标签为空（自我闭合）。没有图像时，它会直接缺省。

6.3 安装文件

有两个标签描述将复制到游戏中的内容：<file> 复制单个文件，<folder> 复制整个文件夹及其所有内容。两者都接受相同的三个属性：

属性	作用	示例
source	该元素在您模组存档中的路径，相对于根。	tex\2k
destination	该元素必须落到的位置，相对于游戏的 Data 文件夹。	textures
priority	发生冲突时的安装顺序（见下文）。	0

priority
属性及其影响。它是一个数字，用于设定当两个选项将文件安装到同一位置时的复制顺序。规则是：低优先级先安装，高优先级后安装——
而由于后写入者获胜，较高优先级会“覆盖”较低优先级。示例：如果一个基础选项以 priority="0" 安装一个纹理，而一个“高分辨率”选项以 priority="1" 安装同一纹理，那么无论选项的显示顺序如何，最终保留的都是高分辨率版本。只要您没有需要刻意仲裁的冲突，就处处保持 0。

> 注意 —— 目标默认为空，表示将该项安装到游戏 Data 文件夹的根目录。每当该项必须放到别处时，单击“目标文件”单元格输入一个明确的路径（相对于 Data）。当源文件夹已镜像游戏的结构时，空目标是正确的选择。

这些标签出现在三个位置：在 <requiredInstallFiles> 中（始终安装的文件）、在某个选项的 <files> 中（若该选项被选中则安装），以及在某个条件模式的 <files> 中（若条件满足则安装）。

6.4 步骤、组与选项

这是向导的核心。层次结构很简单：一个步骤包含多个组，一个组包含多个选项。

<installSteps order="Explicit"> 汇集所有步骤；order="Explicit" 表示“遵循我所写的顺序”。向导的每个页面都是一个 <installStep name="…">，其 name 属性是显示在页面顶部的标题。

<visible> —— 有条件地显示步骤。一个步骤可仅在某些条件为真时才显示；您将这些条件放在一个 <visible> 标签中（参见 § 6.6）。没有 <visible> 时，步骤始终显示。示例：仅当用户先前勾选了“专家安装”时，才显示一个“高级设置”步骤。

<optionalFileGroups> 包含步骤中的各组。每个 <group name="…" type="…"> 都有一个 name（节标题），尤其还有一个 type 用于设定选择规则——即用户被允许勾选什么：

type	用户看到的内容及可执行的操作
恰好一个（必选）—— SelectExactlyOne	单选按钮：他们必须选择一个且只能一个选项。不能为零，也不能为二。适合互斥变体（单一纹理分辨率）。
至多一个 —— SelectAtMostOne	如上，但他们也可以什么都不选：一个“选或不选”。
任意数量 —— SelectAny	独立的复选框：他们随意勾选，从一个都不选到全选。最常见的情形，用于可叠加的模块。
至少一个 —— SelectAtLeastOne	复选框，但他们必须勾选至少一个才能继续。防止空手离开。
全部（无选择）—— SelectAll	所有选项都被自动勾选并锁定：用户无法取消勾选。用于在描述的同时呈现强制性文件。

在一个组中，<plugins order="Explicit"> 包含各选项，每个选项都是一个 <plugin name="…">（这里的“plugin”一词指向导的一个选项，而非一个 .esp 文件）。其 name 是用户勾选的标签。一个选项由以下部分构成：

- <description>：选中该选项时在右侧显示的文本——请在此清晰描述其效果；
- <image path="…"/>：随描述一同显示的插图（可选）；
- <conditionFlags>：选中该选项时所设置的标志（§ 6.6）；
- <files>：若该选项被选中则安装的文件；

- <typeDescriptor>：该选项的默认状态（§ 6.5）。

6.5 选项的类型及其行为

每个选项都有一个“类型”，用于确定其在向导中的默认状态：是否勾选、是否可编辑。它由 <typeDescriptor> 描述。在简单情形下，它包含单个 <type name="…"/>。五种取值及其效果：

type	在向导中的效果
可选 —— Optional	默认未勾选，可自由勾选和取消勾选。常见情形。
推荐 —— Recommended	默认勾选，但用户可取消勾选。用于建议的选择。
必需 —— Required	已勾选并锁定：无法取消勾选。用于必不可少的组件。
不可用 —— NotUsable	未勾选并锁定：无法勾选。表示在当前上下文中不可用的选项。
可能可用 —— CouldBeUsable	可勾选，但标记为可能有问题（通常带有警告）。

动态类型：<dependencyType>。选项的类型也可根据条件而变化。您不写固定的 <type>，而是写一个包含 <defaultType name="…"/>（默认类型，当无任何条件满足时）和一个 <pattern> 列表的 <dependencyType>：每个模式将条件与一个要在其为真时应用的 <type> 关联起来。

示例：一个“ENB Ultra 预设”选项默认可为 Optional，在用户启用了动态烛光时变为 Recommended，而在磁盘上缺少某个必需文件时变为 NotUsable。正是这一机制让安装程序“变得智能”。完整示例见附录 D。第 7.5 节利用这一机制将“预设”转化为用户仍可调整的默认值。

6.6 标志、依赖与条件安装

这三个概念构成了 FOMOD 条件系统——使安装程序能对用户选择作出反应的机制。

标志 —— <conditionFlags> 和 <flag>。标志是您在选中某个选项时所设置的一个小变量。选项的 <conditionFlags> 标签包含一个或多个 <flag name="…">值</flag>：name 是标志名，标签之间的文本是其值。于是 <flag name="res">4K</flag> 会在该选项被勾选时立即将“res”标志设为“4K”。标志本身没有可见效果：它充当记忆，稍后在别处作为条件被重新使用。

依赖 —— <dependencies operator="…">。依赖是一个要检查的条件。它们被归入 <dependencies>，其 operator 属性说明如何组合它们：And（所有条件都必须为真）或 Or（只需一个即可）。存在两种条件：

条件	它所检验的内容
`<flagDependency flag="…" value="…"/>`	若所命名的标志恰好具有所指明的值则为真。这就是您对先前所做选择作出反应的方式（例如：“res”标志等于“4K”）。
`<fileDependency file="…" state="…"/>`	根据用户游戏中某个文件已存在或不存在的状态而为真或为假。

文件依赖的 state 属性取三个值：

state	含义
Active	文件在游戏中存在且已启用。
Inactive	文件存在但已禁用。
Missing	文件不存在。

这些依赖用于三件事：步骤的可见性（在 <visible> 中）、选项的动态类型（在 <dependencyType> 的某个 <pattern> 中），以及下面的条件安装。

条件安装 —— <conditionalFileInstalls>。此部分置于各步骤之后，仅在条件满足时才安装文件，而无需用户直接勾选任何内容。它包含 <pattern> 元素，每个元素将 <dependencies> 与一个 <files> 关联起来。如果在向导结束时该模式的条件为真，则安装其文件。

与步骤可见性的区别：<visible> 决定是否显示某个页面，而 <conditionalFileInstalls> 则在幕后决定是否安装文件。典型情形：仅当用户勾选了两个特定选项（其标志随后都被设置）时，才自动安装一个兼容性补丁。

> 注意 —— 一个标志必须先由某个选项（通过 <conditionFlags>）设置，之后才能被依赖检验。因此请在各步骤中定义您的标志，然后在条件模式中检验它们。
>
> 注意 —— 设计原则：当一个选项仅代表一个决定——一个预设、一个配置方案、一个“模块”开关——
> 就让它只设置标志，不做其他任何事，并将复制工作交给以这些标志为条件的条件安装。这样，每项功能一个标志，便成为唯一的事实来源：无论选择如何组合，预设和手动选中的选项都绝不会将相同的文件安装两次。第 7.5 节应用了这一原则。

7. ModuleConfig.xml 示例

以下示例说明了最常见的情形。它们对应 XIMOD Architect 实际生成的 XML（为便于阅读，缩进已简化）。

7.1 最简示例 —— 必装文件

一个没有任何选择的模组：所有文件都直接安装。

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>My Simple Mod</moduleName>
    <requiredInstallFiles>
        <file source="MyMod.esp" destination="MyMod.esp" priority="0"/>
        <folder source="textures" destination="textures" priority="0"/>
    </requiredInstallFiles>
</config>
```

7.2 互斥选择 —— SelectExactlyOne

一个提供单一纹理分辨率选择的步骤；2K 选项默认推荐。

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>HD Textures</moduleName>
    <moduleImage path="fomod\header.png"/>
    <installSteps order="Explicit">
        <installStep name="Texture resolution">
            <optionalFileGroups>
                <group name="Resolution" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="2K">
                            <description>2K textures (balanced).</description>
                            <files>
                                <folder source="2k" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Recommended"/>
                            </typeDescriptor>
                        </plugin>
                        <plugin name="4K">
                            <description>4K textures (high quality).</description>
                            <files>
                                <folder source="4k" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Optional"/>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
    </installSteps>
</config>
```

7.3 条件安装 —— 标志

一个选项在被勾选时设置一个标志；仅当该标志处于活动状态时才安装某个文件。

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Conditional patch</moduleName>
    <installSteps order="Explicit">
        <installStep name="Options">
            <optionalFileGroups>
                <group name="Modules" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Module A">
                            <description>Enables module A.</description>
                            <conditionFlags>
                                <flag name="ModuleA">On</flag>
                            </conditionFlags>
                            <files>
                                <folder source="A" destination="meshes" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Optional"/>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
    </installSteps>
    <conditionalFileInstalls>
        <patterns>
            <pattern>
                <dependencies operator="And">
                    <flagDependency flag="ModuleA" value="On"/>
                </dependencies>
                <files>
                    <file source="patchA.esp" destination="patchA.esp" priority="0"/>
                </files>
            </pattern>
        </patterns>
    </conditionalFileInstalls>
</config>
```

7.4 一个页面，多个步骤 —— SelectAny + 标志

单个多选步骤可为玩家勾选的每个选项打开一个不同的步骤：步骤并不与选项绑定。每个选项设置一个条件标志，每个目标步骤对该标志携带一个可见性条件。于是勾选 A 和 B 会显示步骤 A 和 B；勾选 A、B 和 C 会显示三个步骤。

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Modular Add-ons</moduleName>
    <installSteps order="Explicit">
        <!-- Step 1: the choices. Each ticked option sets its own flag. -->
        <installStep name="Choose modules">
            <optionalFileGroups>
                <group name="Modules" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Module A">
                            <description>Enable module A.</description>
                            <conditionFlags>
                                <flag name="want_A">on</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="Module B">
                            <description>Enable module B.</description>
                            <conditionFlags>
                                <flag name="want_B">on</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <!-- Modules C and D follow the same pattern (want_C, want_D). -->
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step A: shown only when want_A is on. -->
        <installStep name="Module A options">
            <visible>
                <flagDependency flag="want_A" value="on"/>
            </visible>
            <optionalFileGroups>
                <group name="Module A" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="A - extra textures">
                            <description>Optional high-res textures for A.</description>
                            <files>
                                <folder source="a_textures" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step B: shown only when want_B is on. -->
        <installStep name="Module B options">
            <visible>
                <flagDependency flag="want_B" value="on"/>
            </visible>
            <optionalFileGroups>
                <group name="Module B" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="B - extra meshes">
                            <description>Optional meshes for B.</description>
                            <files>
                                <folder source="b_meshes" destination="meshes" priority="0"/>
                            </files>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Steps for C and D have exactly the same shape, each guarded by its
             own <visible><flagDependency flag="want_C | want_D" value="on"/>. -->
    </installSteps></config>
```

7.5 带可自定义默认值的预设

“预设”是一个同时代表多项选择的选项。用户两者都想要：选择预设后就此完成，或者选择预设后仍调整一两项功能。关键在于将决定与安装分开。第一页负责决定——
它设置标志。按需显示的功能页面让用户手动选择——
它们设置另外的标志。条件安装则根据这些标志执行复制。

步骤 1
提供预设，并为每项功能提供一个双选项组：“沿用预设的选择”或“自定义”。第二个选项会打开该功能的页面，该页面仅在其“自定义”标志为 on 时可见（§ 7.4）。在该页面上，选项使用动态类型（§ 6.5）：所选预设本应安装的变体为 Recommended——即预先勾选——
否则推荐“无”，因此用户可以保留预设的变体、切换到另一个变体，或移除该功能。

预设和页面选项都不包含任何 <files>。所有复制都在 <conditionalFileInstalls> 中进行，每个选项对应一个模式：当该功能已自定义且该选项在其页面上被勾选时，或者当该功能交由预设决定且该预设选择了此选项时，其文件才会被安装。正是这一点使重复安装成为不可能：对每项功能而言，两个分支中恰好只有一个成立，因而只有一组文件。

> 注意 ——
> 一个标志，一个写入者。模组管理器将标志保存在一张每次点击都会更新的表中：选中一个选项会写入它的标志，取消选中则将其移除，最后写入者胜出。如果两个可以同时选中的选项写入同一个标志——一个同时设置某页面标志的预设、一个重置这些标志的“自定义”复选框——
> 那么后退并再次点击其中一个，就会悄无声息地覆盖另一个仍显示为选中的内容。因此，请让每个标志只由一组互斥的选项写入：preset 由“预设”组写入，cust_A 由功能 A 的模式组写入，feat_A 由页面 A 的选项写入。这样，无论点击顺序如何，标志始终与用户所见一致。

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>带可自定义默认值的预设</moduleName>
    <installSteps order="Explicit">
        <!-- 步骤 1：预设写入 preset 标志，除此之外什么也不做。 -->
        <installStep name="安装模式">
            <optionalFileGroups>
                <group name="预设" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="预设 A（推荐配置）">
                            <description>以版本 2 安装功能 A，以版本 1 安装功能 B。</description>
                            <conditionFlags><flag name="preset">A</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="无预设（自行搭配）">
                            <description>手动选择每项功能。</description>
                            <conditionFlags><flag name="preset">none</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
                <!-- 每项功能一个组：只有它写入该功能的“自定义”标志。 -->
                <group name="功能 A" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="沿用预设的选择">
                            <description>按预设的方式安装功能 A。</description>
                            <conditionFlags><flag name="cust_A">off</flag></conditionFlags>
                            <typeDescriptor><type name="Recommended"/></typeDescriptor>
                        </plugin>
                        <plugin name="自定义">
                            <description>打开功能 A 的页面。</description>
                            <conditionFlags><flag name="cust_A">on</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
```

```
                    </plugin>
                </plugins>
            </group>
            <!-- “功能 B”、“功能 C”...拥有相同的两个选项，并写入 cust_B、cust_C。 -->
        </optionalFileGroups>
    </installStep>
    <!-- 步骤 2：功能 A，仅在用户要求自定义它时显示。 -->
    <installStep name="功能 A">
        <visible><flagDependency flag="cust_A" value="on"/></visible>
        <optionalFileGroups>
            <group name="功能 A" type="SelectExactlyOne">
                <plugins order="Explicit">
                    <plugin name="无">
                        <description>不安装功能 A。</description>
                        <conditionFlags><flag name="feat_A">none</flag></conditionFlags>
                        <typeDescriptor>
                            <dependencyType>
                                <defaultType name="Recommended"/>
                                <patterns>
                                    <pattern>
                                        <dependencies operator="And">
                                            <flagDependency flag="preset" value="A"/>
                                        </dependencies>
                                        <type name="Optional"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                    <plugin name="版本 1">
                        <description>功能 A，版本 1。</description>
                        <conditionFlags><flag name="feat_A">v1</flag></conditionFlags>
                        <typeDescriptor><type name="Optional"/></typeDescriptor>
                    </plugin>
                    <plugin name="版本 2">
                        <description>功能 A，版本 2（预设的选择）。</description>
                        <conditionFlags><flag name="feat_A">v2</flag></conditionFlags>
                        <typeDescriptor>
                            <dependencyType>
                                <defaultType name="Optional"/>
                                <patterns>
                                    <pattern>
                                        <dependencies operator="And">
                                            <flagDependency flag="preset" value="A"/>
                                        </dependencies>
                                        <type name="Recommended"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                </plugins>
            </group>
        </optionalFileGroups>
    </installStep>
    <!-- 功能 B 和 C 的页面结构完全相同。 -->
</installSteps>
<!-- 文件仅在此处复制：功能已自定义时由页面复制，否则由预设复制。 -->
<conditionalFileInstalls>
    <patterns>
        <pattern>
            <dependencies operator="And">
                <flagDependency flag="cust_A" value="on"/>
                <flagDependency flag="feat_A" value="v1"/>
            </dependencies>
            <files><folder source="FeatureA\v1" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="Or">
                <dependencies operator="And">
                    <flagDependency flag="cust_A" value="on"/>
                    <flagDependency flag="feat_A" value="v2"/>
                </dependencies>
                <dependencies operator="And">
                    <flagDependency flag="cust_A" value="off"/>
```

```
                    <flagDependency flag="preset" value="A"/>
                </dependencies>
            </dependencies>
            <files><folder source="FeatureA\v2" destination="" priority="0"/></files>
        </pattern>
        <!-- 功能 B 同理：v1 由页面或预设 A 安装。 -->
    </patterns>
</conditionalFileInstalls>
</config>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>带可自定义默认值的预设</moduleName>
    <installSteps order="Explicit">
        <!-- 步骤 1：预设设置其所包含的每项功能的标志，除此之外什么也不做。 -->
        <installStep name="安装模式">
            <optionalFileGroups>
                <group name="预设" type="SelectAtMostOne">
                    <plugins order="Explicit">
                        <plugin name="预设 A（推荐配置）">
                            <description>以版本 2 安装功能 A，以版本 1 安装功能 B。</description>
                            <conditionFlags>
                                <flag name="preset">on</flag>
                                <flag name="feat_A">v2</flag>
                                <flag name="feat_B">v1</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
                <group name="自定义" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="自定义功能 A">
                            <description>打开功能 A 的页面。</description>
                            <conditionFlags><flag name="cust_A">on</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <!-- “自定义功能 B”、“自定义功能 C”...设置 cust_B、cust_C。 -->
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- 步骤 2：功能 A，仅在用户要求自定义它时显示。 -->
        <installStep name="功能 A">
            <visible><flagDependency flag="cust_A" value="on"/></visible>
            <optionalFileGroups>
                <group name="功能 A" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="无">
                            <description>不安装功能 A。</description>
                            <conditionFlags><flag name="feat_A">none</flag></conditionFlags>
                            <typeDescriptor>
                                <dependencyType>
                                    <defaultType name="Recommended"/>
                                    <patterns>
                                        <pattern>
                                            <dependencies operator="And">
                                                <flagDependency flag="preset" value="on"/>
                                            </dependencies>
                                            <type name="Optional"/>
                                        </pattern>
                                    </patterns>
                                </dependencyType>
                            </typeDescriptor>
                        </plugin>
                        <plugin name="版本 1">
                            <description>功能 A，版本 1。</description>
                            <conditionFlags><flag name="feat_A">v1</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="版本 2">
                            <description>功能 A，版本 2（预设的选择）。</description>
                            <conditionFlags><flag name="feat_A">v2</flag></conditionFlags>
                            <typeDescriptor>
                                <dependencyType>
```

```
                                <defaultType name="Optional"/>
                                <patterns>
                                    <pattern>
                                        <dependencies operator="And">
                                            <flagDependency flag="preset" value="on"/>
                                        </dependencies>
                                        <type name="Recommended"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                </plugins>
            </group>
        </optionalFileGroups>
    </installStep>
    <!-- 功能 B 和 C 的页面结构完全相同。 -->
</installSteps>
<!-- 文件仅在此处复制，依据每个功能标志的最终值。 -->
<conditionalFileInstalls>
    <patterns>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_A" value="v1"/></dependencies>
            <files><folder source="FeatureA\v1" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_A" value="v2"/></dependencies>
            <files><folder source="FeatureA\v2" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_B" value="v1"/></dependencies>
            <files><folder source="FeatureB\v1" destination="" priority="0"/></files>
        </pattern>
    </patterns>
</conditionalFileInstalls>
</config>
```

项目 → 预览安装程序… 让您可逐一走查每种组合——仅预设、预设加“自定义功能 A”、仅“自定义功能 A”——并在发布前检查最终文件列表（§ 4.11、§ 8.19）。项目 → 验证 FOMOD 会报告相互冲突的目标，并能识别这种预设 / 自定义模式（§ 8.1）。

在安装程序中后退。模组管理器会保留用户已看过的页面上的选择：如果用户返回第一页并选择另一个预设，已访问过的功能页面会保留之前的勾选，而不是采用新预设的默认值。只要每个标志只有一个写入者，这就无伤大雅：标志始终与屏幕一致，因此用户最终看到的内容——交由预设决定的功能的预设、已自定义页面上的勾选——正是实际安装的内容。预设向导（§ 8.29）会构建这种结构；由早期版本构建的结构（其中预设还会写入页面的标志）可通过再次运行向导来重建。

7.6 欢迎页与结束页

向导由模组管理器绘制，而非由安装程序绘制。作者能控制步骤、组和选项的名称、描述和图像——仅此而已：没有字体、没有颜色、没有布局，也没有自由文本块。在这些限制之内，欢迎页和结束页很容易实现。

欢迎页——将第一个步骤设为仅含一个 SelectAll 类型的组，其中只有一个 Required 选项。其描述承载欢迎文字，由管理器显示在描述区域中；其图像承载视觉内容——横幅、徽标，或以您喜欢的任意字体渲染成图片的文字。不要为它分配任何文件；如果您希望模组的核心文件在此列出，也可以为它分配这些文件。

结束页——在最后一个步骤采用同样的构造：一个 Required 选项，附上告别信息，必要时说明后续操作（加载顺序、需要安装的补丁）。安装程序无法显示用户所选内容的摘要：该格式不允许一个页面访问在其他页面上所做的选择。管理器会显示其自

身的待安装文件列表，而安装程序预览（§ 4.11）则为身为作者的您提供每种场景下的这份摘要。

```
<installStep name="欢迎">
    <optionalFileGroups>
        <group name="关于此安装程序" type="SelectAll">
            <plugins order="Explicit">
                <plugin name="欢迎使用 Dawnlight">
                    <description>感谢您下载
Dawnlight。接下来的页面可让您选择光照预设、可选模块和兼容性补丁。在最后一页之前不会安装任何内容。</description>
                    <image path="fomod\welcome.png"/>
                    <typeDescriptor><type name="Required"/></typeDescriptor>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>
<!-- ... 实际步骤 ... -->
<installStep name="全部完成">
    <optionalFileGroups>
        <group name="开始游戏之前" type="SelectAll">
            <plugins order="Explicit">
                <plugin name="谢谢！ ">
                    <description>请将 Dawnlight.esp 放在您的天气模组之后。如有任何问题，请在 Nexus
页面上反馈。祝您游戏愉快！</description>
                    <image path="fomod\thanks.png"/>
                    <typeDescriptor><type name="Required"/></typeDescriptor>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>
```

将欢迎步骤保存为模板（项目 → 模板…，§ 8.3），即可在每个模组中重复使用。

8. 版本 2 —— 新功能

以下功能由版本 2.0.0 引入。它们按添加的顺序呈现；第 2 至 7
章中带编号的小节已反映它们。第 8.26 至 8.29 节为版本 2.1.0 新增的内容。第 8.30 至 8.32
节为版本 2.2.0 新增的内容。

8.1 目标冲突检测

当您验证项目（项目 → 验证 FOMOD）时，XIMOD
现在还会报告将不同文件安装到同一目标路径（会相互覆盖）的选项。可能同时发生的冲突会被标记为“确定”；仅在互斥选项之间才会出现的冲突会被标记为“可能”。
仅当两个选项确实可同时被选中时，冲突才会被报告为“确定”。除互斥组（SelectExactlyOne / SelectAtMostOne）外，XIMOD
现在还会追踪标志：如果一个选项将某个标志设为某一值，而另一个选项位于一个仅在该标志为另一值时才显示的页面上——且在第一个选项旁可同时选中的任何选项都无法设置那另一值——则这一对无法共存，该冲突仅会被列为“潜在”。典型情形：一个“预设”选项与一个“自选”页面中相匹配的选项。

8.2 7z 导出

导出命令现在除 ZIP 外还提供 7-Zip（.7z）。在保存对话框中选择格式；7z
通常会生成更小的下载文件。

8.3 可复用模板

项目 → 模板…
让您可将所选步骤保存为可复用模板，并将已保存的模板（步骤、组或选项）插入任何项目。模板存储在应用的配置文件夹下。

8.4 从文件夹新建

文件 → 从文件夹新建…
会扫描一个文件夹，并提出一个框架，其中每个直接子文件夹成为一个选项；位于根部的明显页眉图像会被自动检测。

8.5 拖放式文件分配

在树中选中某个选项后，从文件资源管理器（Windows 资源管理器、Finder、Linux
文件管理器）将文件或文件夹拖放到 XIMOD
窗口上：它们会被添加到该选项的“源文件”表格中，与使用“添加文件”/“添加文件夹”完全相同，并且可以一次拖放多个项目。只接受位于项目根文件夹下的路径；其他路径会被忽略。对于大型安装程序，这是分配来源的最快方式：在资源管理器中打开一次功能文件夹，然后将每个子文件夹拖放到其对应的选项上。

8.6 FOMOD 比较

项目 → 与…比较 会将当前项目与另一个 FOMOD
进行比较，并列出新增、移除或更改的内容（步骤、选项、文件和元数据）——
便于编写更新日志。

8.7 可视化条件编辑器

项目 → 条件编辑器…
会为每个条件标志显示哪些选项设置了它，以及哪些步骤可见性规则、条件安装和依赖模式使用了它。被设置但从未使用（或被使用但从未设置）的标志会被高亮显示。

8.8 图像检查与优化

验证现在会对过大、格式不受支持或无法读取的页眉/选项图像发出警告。在“模组信息”标签页中，“优化图像”会将页眉图像缩小到合理的范围。

8.9 命令行新增项

命令行新增：inspect <plugin.esp>（打印插件的主文件、作者和 ESL 标志）；batch <manifest.json>（验证、构建并打包多个 FOMOD，用于持续集成）；以及 ba2 <dir> <out.ba2> [game]，将一个文件夹打包为通用 BA2 存档。BA2 打包尚处于实验阶段——
发布前请在游戏内验证该存档。

8.10 后台导出与验证

导出分发存档不再使窗口冻结：FOMOD
写入后，压缩在后台运行，一个进度窗口会显示已处理文件的数量和正在添加的文件。一个“取消”按钮可停止导出并移除部分完成的存档。同样，验证的磁盘检查（缺失文件、目标冲突、图像）在后台运行；报告窗口会显示“正在检查磁盘上的文件…”直至它们完成，并在此期间项目若有变化会向您发出警告。

8.11 崩溃恢复

如果 XIMOD Architect
意外关闭，每个有未保存更改的项目都会被写入应用配置目录下的一个恢复文件夹（crash_recovery/<日期>/<模组名称>/fomod/）。在下次启动时，状态栏会显示备份保存的位置；用 文件 → 打开文件夹… 打开该文件夹即可找回您的工作。

8.12 更清晰的界面

一些日常的烦恼得到了解决。保存一个尚无文件夹的项目时，现在会询问将其保存到何处，而不是什么都不做，并且 文件 →
另存为…（Ctrl+Shift+S）可将项目移动到另一个文件夹。当有未保存的更改时，窗口标题会显示模组名称和一个 ●
圆点。成功、警告和错误会以通知形式出现在右上角（错误会一直保留直到关闭），而状态栏则保留最后一条消息以及步骤和选项的数量（根文件夹显示在检查器的“工作区”部分，模组名称显示在窗口标题和文档标签页中）。选择根文件夹之外的文件，现在会被报告而不再被悄然忽略。
术语 —— 界面现在在所有语言中一律将 FOMOD 格式所称的
plugin（一个组中提供的选择）称为“选项”；“插件”一词保留给游戏插件文件（.esp、.esm、.esl）。主题设置“跟随系统”会实时跟随操作系统：在浅色和深色模式之间切换操作系统会立即切换 XIMOD，无需重启，且所选的字体大小在两者中都保留。最后，各数据集（游戏、国家/地区、语言）现在在启动画面显示期间读取，而非在其之后，因此主窗口出现得更快。

8.13 翻译现有的 FOMOD

项目 → 翻译 FOMOD…
会打开一个专用窗口，用于将安装程序面向用户的字符串翻译为另一种语言：模组名称和描述，步骤、组和选项的名称，以及选项描述。标志、文件路径和类型绝不会被触碰，因为模组管理器会按字面对它们进行比较。加载活动项目（它必须已保存），或用“打开一个模组文件夹…”加载

任意模组文件夹；选择源语言和目标语言，然后填写“译文”列，对名称可直接在表格中填写，对描述则在下方面板中填写（源文本并排显示，空格和换行可选择性地可见）。筛选器（未翻译、待审校、有问题、已锁定）、一个搜索框、一个进度条以及
Ctrl+Enter（下一个未翻译项）有助于跟踪进度。作者和网站默认锁定：已锁定的字符串会原样写入。
问题会在每一行上标出：翻译中缺失的受保护标记（文件名、URL、字面的 \n
序列）、名称中的换行、XML
无法存储的字符、与源文本相同的翻译，或同一源文本在别处被以不同方式翻译。阻断性问题会阻止导出。工作会保存（Ctrl+S）为一个附属文件 fomod/translations/<模组>.<语言>.ximod-translation，下次打开该文件夹时该窗口会自动重新加载它；“从文件夹更新”会重新读取 FOMOD 并报告新增、更改、移动和移除的字符串，同时保留现有翻译（已更改的源文本会被标记为“待审校”）。
“导出翻译后的文件”会将 info.xml 和 ModuleConfig.xml 写入一个同级文件夹
fomod_<语言>（默认，原文件不受影响），或在生成一份带时间戳的 .bak
副本后覆盖原文件。导出是无损的：仅翻译后的字符串发生变化；原 XML 的注释、编码（UTF-8 或 UTF-16）、未知元素和格式都会逐字节保留。“保持原始顺序”会强制
order="Explicit"，以免按名称排序的列表因译名而被重新排序。“预览翻译后的版本”会在翻译后的副本上打开安装程序模拟。命令行上也有相同的操作：ximod-architect translate extract | status | update | apply（参见 § 5）。
翻译包 —— 第三种导出模式会构建一个可直接上传到 Nexus
的存档：默认为一个“仅补丁”包，仅包含翻译后的 info.xml 和
ModuleConfig.xml（位于其原始路径和编码），外加一份目标语言的
README，用户将其安装到原模组之上；“完整模组”选项则改为包含模组的每个文件（请核实作者是否允许再分发）。格式（.zip 或 .7z）和文件名遵循“名称”模板，可使用标记
{name}、{version}、{LANG}、{lang}、{lang3} 和 {langname}（默认“{name}_{LANG}”，例如 MyMod_FR.zip）。存档在后台构建，带有通常的进度窗口。
效率 ——
一个按语言对存储于配置文件夹中的翻译记忆库会学习您保存的每一条翻译；“从记忆库填充”会在下一个模组中预填相同的字符串（标记为“待审校”），下方面板会为所选字符串建议所记忆的翻译。“传播到相同项”会将当前翻译复制到所有具有相同源文本的其他字符串。“术语表”（工具行中的切换开关）列出强制术语——
一个术语及其期望翻译，可选择区分大小写，或一个需保持不翻译的术语——
任何违反规则的行都会被标记为问题。“导出 CSV” / “导入
CSV”可与电子表格或另一位翻译者交换整张表格（键、上下文、源、翻译、状态、备注）；导入会按键匹配行，并能容忍 Excel 的分隔符。
唯一文本 —— 筛选行中的一个切换开关会为每个不同的源文本列出一行，并带有一个 ×N
徽章标明该组的大小；翻译该行会一次性自动翻译所有具有相同文本的字符串，无需询问（已锁定的字符串除外），一条消息会说明更新了多少条。对于在许多页面上重复相同选项名称或描述的
FOMOD，这是最快捷的处理方式。关闭时，表格会显示每个字符串，“传播到相同项”仍可用于一次性仅复制到那些未翻译项。
键盘与空白 —— 在表格中，↑ 和 ↓ 移动所选行，Page Up 和 Page Down 跳过一屏的行，Home 和 End
转到第一行和最后一行（在未输入任何翻译字段时；相同的键在“项目字符串”窗口中也有效）。“显示空白和换行”会使源文本中的每个空白可见，无论是在表格中还是在详情面板中：·
表示空格，␣ 表示不换行空格，→ 表示制表符，␍ 表示回车，¶ 位于每行末尾。
命令行上：ximod-architect translate package <root> <sidecar> [-o OUT] [--format zip|7z] [--full]、translate export-csv / import-csv、translate tm-learn / tm-apply；并且一个 batch
清单条目可携带一个“translations”列表（sidecar、mode、out、format、full），以便在生成主存档的同时生成翻译包。

8.14 打开模组存档

文件 → 打开压缩包…（或将一个 .zip/.7z
文件拖放到窗口上）会在后台将存档解压到配置目录的工作文件夹（work/<存档名称>）中，带有通常的进度窗口，然后打开其中找到的 FOMOD——无论 fomod
文件夹位于存档顶部还是位于单个包裹文件夹之下。已经解压过的存档会被原样重用，因此您在那里所做的编辑会被保留。不支持 .rar 存档：请先用 7-Zip
解压。翻译窗口有相同的按钮“打开一个模组存档…”，它只解压 fomod 文件夹（解压到 translate_work/<存档名称>）；此时“完整模组”包选项不可用，因为模组文件不在那里。
命令行上：ximod-architect extract <archive> [-o DIR] [--fomod-only]
会解压一个存档（默认解压到一个以其命名的文件夹），并打印 FOMOD 根。

8.15 导入保真度

XIMOD
现在会读取并回写它过去会悄然丢弃的结构：模组级的依赖要求（moduleDependencies，在“模组信息”标签页中的“模组要求”下用与别处相同的依赖编辑器进行编辑）、文件和文件夹的 alwaysInstall 和 installIfUsable
属性（文件表格的“始终”和“可用时”列）、标题位置和颜色，以及页眉图像选项 showImage、showFade 和 height（模组信息 → “高级页眉”），还有步骤、组和选项的 Ascending/Descending 顺序属性，它们会按作者所设保留。
仍然无法建模的内容——架构之外、XIMOD 不认识的元素——会在加载项目时报告：一条警告“此 FOMOD 包含 N 个 XIMOD
无法编辑的结构；它们将在项目保存时被丢弃”会打开“问题”面板，每个结构及其位置（步骤、选项、条件集）占一行；单击某行会选中相应节点。只要项目保持打开，这些警告便会在各次验证间始终留在“问题”面板中。ximod-architect validate 会打印相同的警告。

8.16 备份与恢复

每当一个项目覆盖保存到一个现有版本时，先前的 info.xml 和 ModuleConfig.xml 会首先被复制到 fomod/backups/<日期-时间>/
中（当它们与即将写入的内容完全相同时则不复制），并且只保留最近的 N 个文件夹——N 为“保留的备份数”设置。该文件夹绝不会被包含在分发存档或 BA2
包中，其文件也绝不会被报告为孤立文件。两个 XML
文件都以原子方式写入（临时文件、刷新、重命名），因此保存期间的崩溃不会留下一个只写了一半的安装程序。
文件 → 恢复备份…
会列出各备份，最新的在前，每个都标明与当前项目之间的差异数量；“比较”会针对该备份打开通常的比较窗口，“恢复”会将它作为一次可撤销的更改加载到编辑器中（在您保存之前不会向磁盘写入任何内容），而“删除所有备份”——由第二次单击确认——会清空该文件夹。除 § 8.11 的崩溃防护外，现在还会按固定间隔写入每个已修改项目的恢复副本（“自动保存恢复副本的间隔（分钟）”）；它会在正常退出时被清除，因此恢复提示仍然只在异常退出后才出现。
命令行上：ximod-architect backups <root> --list、--restore <timestamp> 或 --prune N；build 和 package 在写入前也会生成一个备份。

8.17 将主文件作为自动条件

当一个插件文件（.esp、.esm 或 .esl）被添加到某个选项时——无论从文件表格还是通过拖放——XIMOD
都会读取其标头：如果项目尚无作者，则填入该插件的作者；并且该插件所需要的每个主文件，只要它既不是游戏自身的文件（在 Categories.json
中按游戏列于“masters”下）也不是本模组安装的文件，就会成为该选项的一个 File
条件（“Active”），并被添加到该选项默认类型的第一个依赖模式中（如有需要则创建它）。已

被该选项某个 File
条件命名的主文件绝不会被添加两次。设置“将插件的主文件添加为条件”可将此关闭。
验证会读取项目所安装的每个插件（必装文件、选项、条件集）的标头，并对任何既不在模组中、也不是游戏基础文件、也未声明为依赖的主文件发出警告：“X 需要 Y，而 Y
既不在本模组中也未声明为依赖”。该警告指向相应节点，因此在“问题”面板中单击它会选中该选项。

8.18 轻量插件（ESL）与插件报告

验证还会检查轻量插件规则：一个 .esl 扩展名但其 light 标志未设置的文件、一个被标记为 light
但其新建记录超过上限或落在允许的 FormID 范围之外的插件，以及——作为信息——
一个可被标记为 light 的 .esp（它会统计该插件新建的记录数，即那些 FormID
索引等于主文件数量的记录，并与游戏的上限比较：默认 2048，Starfield 为 4096，可通过 Categories.json 中的“eslLimit”按游戏调整）。超过 64 MB 的文件不计入。CLI 命令 inspect <plugin> 会打印相同的数据。
工具 → 插件报告… 会将这一切汇集到一张表格中：文件、类型、light
标志、主文件（缺失的以警告色显示）、相对于上限的新建记录数，以及资格；无法读取的文件会如此标注。

8.19 更丰富的模拟器与场景

安装程序预览（项目 → 预览）的摘要页面现在以一个“最终文件树”结尾：将落入游戏 Data
文件夹的每个文件、从磁盘展开的文件夹及其大小，以及总安装大小。当两个条目落到同一最终路径时，优先级较高者获胜（相同时以较晚的声明为准），失败者会以删除线显示，悬停时显示获胜的选项。超过 20000 个条目的树会被截断并予以说明。
文件树遵循与 XIMOD
中每棵树相同的规则。在文件夹上右键单击会打开“展开”和“折叠”子菜单（全部 / 选中 /
从选中开始，最后一个作用于该文件夹及其下的所有内容，包括已安装存档的内容）。当没有文本框获得焦点时，↑ 和 ↓ 使光标环在各行之间移动，Home 和 End 跳到两端，→
打开一个文件夹（或步入一个已打开的文件夹），← 关闭它（或从一个文件返回到它），Enter
在主窗口中选中安装该行的组件：选项、必装文件或条件集——
对于文件夹，则是其第一个文件所属的组件。
预览还会像模组管理器那样，在第一个步骤之前打开一个“模组信息”页面：页眉图像（遵循 showImage 和 height 选项）、带颜色的标题、作者、版本、网站链接以及来自 info.xml
的描述。“下一步”会引导至第一个可见步骤，“上一步”会返回它，进度条会将其计为第一页。
“场景”行让您可将当前的选择和文件假设以一个名称保存（存储在 fomod/scenarios/<名称>.json 中，以选项名称存储，以便在重新排序后仍然有效），并可重新加载或删除它们；某个场景的选择若不再匹配任何选项，会报告有多少项未找到。场景文件夹绝不会被包含在分发存档中。
验证现在还会作为信息报告那些永远无法到达的步骤、选项和条件集，因为它们检验的某个标志值没有任何选项设置（对步骤而言：更早的步骤中没有任何选项）。
命令行上：ximod-architect simulate <root> [--scenario <name|file.json>] [--json]
会打印可见步骤、各项选择、带有大小和覆盖情况的最终安装列表以及总计；一个 batch
清单条目可携带“scenarios”: [...] 以重放它们（当某个场景不再匹配时该条目会失败）。

8.20 每个选项的安装大小

在一次验证之后（它会在后台测量文件），每个选项都会在其文件表格上方显示其安装大小和文件数量——缺失的来源以警告色计入——
并且项目树会在选项名称之后带上该大小。大小按文件修改时间缓存，并在每次验证时刷新。ximod-architect validate <root> --sizes 会打印各选项、必装文件和条件集的总计。

8.21 Nexus 描述生成器

工具 → Nexus 描述… 会根据项目构建模组 Nexus
页面的文本：标题和描述、“前置要求”（所安装插件中游戏和模组都未提供的主文件，加上所声明的文件依赖）、“安装选项”（每个步骤一个项目符号，组带其选择类型，选项带其描述的第一句话）、一条安装说明，以及相对于用“先前版本…”所选的 FOMOD
的先前版本计算出的“更新日志”（新增或移除的步骤、选项和文件，元数据更改）。选择 BBCode（Nexus）或
Markdown，勾选您想要的部分，在项目中找到的翻译附属文件中选择语言，然后“复制”或“另存为…”。命令行上：ximod-architect nexus-desc <root> [--format bbcode|md] [--previous <root>] [--lang <code>] [-o FILE]。

8.22 可编辑的条件编辑器

§ 8.7
的条件编辑器现在可作用于项目。每一行“设置者”和“使用者”都是一个链接，可在主窗口中选中相应节点。每个标志都有“重命名…”（一个内联字段；新名称会作为一次可撤销的更改应用到设置该标志的每个选项，以及检验它的每个步骤可见性、选项模式、条件集和模组依赖要求；已被另一个标志使用的名称会被拒绝）和“删除所有用法”。在每个标志下，会列出它被设成的各个值以及它被检验时的各个值：一个被检验但从未被设置的值以警告色显示，一个被设置但从未被检验的值以灰色显示。验证会报告相同的结果——
对于没有任何选项设置的被检验值发出警告（指向检验它的节点），对于被设置但从未被检验的标志发出信息。
窗口底部的“条件构建器”会编辑主窗口中所选节点的条件——
一个步骤的可见性、一个选项的依赖模式（选择模式、创建一个、更改其类型）、一个条件文件集或模组依赖要求。它会将该条件显示为一句话（如果 标志 A = B 且 文件 C 为 Active 则 …），上方每个条件占一行：类型（flag 或
file）、名称和值（带有取自项目的建议：已知标志及其值、文件名、Active/Inactive/Missing）、一个移除按钮以及“添加条件”。更改会实时应用且可撤销。
该编辑器现在是一个独立窗口：它可被移到主窗口旁或移到第二块屏幕上，会在各次会话之间保持其位置和大小，并以其标题栏按钮关闭。标志列表的工作方式与项目树相同。在标志标题上右键单击会打开“展开”（全部 / 选中）和“折叠”（全部 /
选中）子菜单。当没有文本框获得焦点时，↑ 和 ↓ 使光标环在各行之间移动——
标志标题及其“设置者” / “使用者”行——，→
展开一个已折叠的标志（或进入一个已展开的标志），←
折叠一个已展开的标志（或从其某一行返回到它），Enter
在检查器中显示该行：某一行的节点，或某个标志的第一个站点。在项目树中，Enter
也是如此：它选中光标下的行，检查器随之跟进。

8.23 项目字符串表格

工具 → 项目字符串…
会将项目的每个名称和描述（模组信息、步骤、组、选项）列入一张表格：位置、字段和文本，文本可就地编辑（名称为单行，描述为多行）。一个搜索字段、一个种类筛选器（全部、名称、描述）和一个“仅重复项”切换开关可缩小行范围；相同的文本（忽略大小写和空白）会被标上 ×N 徽章，悬停时显示其他位置，这有助于发现复制粘贴的描述。查找/替换行（区分大小写、全字匹配）可在当前行或每个可见行中进行替换。双击某个位置会在主窗口中选中相应节点。命令行上：ximod-architect strings <root> [--duplicates] [--csv FILE]。

8.24 读取 BSA 和 BA2 存档

XIMOD 现在无需解包即可读取 Bethesda 存档的目录表：BSA 版本
103（Oblivion）、104（Skyrim、Fallout 3/New Vegas）和 105（Skyrim Special Edition），以及

Fallout 4 和 Starfield 的 BA2 通用（GNRL）和纹理（DX10）存档（版本 1、2、3、7 和 8）。Morrowind
存档会被拒绝并给出清晰的消息。仅读取条目名称和大小；压缩的条目会以其打包大小列出。这为三处提供数据。目标冲突检测（§
8.1）现在会查看某个选项所安装的存档内部：两个选项的存档包含相同资产时会被报告为警告（“由游戏的存档加载顺序决定”），而一个既被打包又以散装文件安装的资产则作为信息报告（“散装文件获胜”）。模拟器的最终文件树（§
8.19）会在每个已安装存档下列出其内容，标注为“（存档内）”，总计会显示有多少内容被打包在存档中（且不会重复计算）。而 工具 →
压缩包内容…（或文件表格中某个存档旁的眼睛按钮）会打开一张条目表格——
格式、数量、解包后总大小、路径、大小、压缩情况——带有一个搜索字段。命令行上：ximod-architect archive <file.bsa|file.ba2> [--json]；validate 现在还会打印确定的目标冲突。
实验性的 BA2 写入器（CLI
ba2）现在会通过用新的读取器回读其输出来验证：路径、大小以及名称/文件夹哈希都相符。

8.25 嵌套条件组与版本条件

条件不再局限于带单个运算符的一张扁平列表。每个持有条件的位置——
一个步骤的可见性、一个选项的依赖模式、一个条件文件集以及模组依赖要求——
现在都是一个可包含条件和嵌套组的组，每个组都有各自的 And / Or
运算符，可达任意深度，完全符合 FOMOD 架构所允许的那样：“(DataVersion = Standard OR DataVersion = Undelayed) AND DataPreset =
Aurelia”会按所写内容保留、加载、编辑和保存而不变化（已在 Aurelia UI 及其 234
个条件集上验证）。还新增了两种条件类型：游戏版本（gameDependency）和模组管理器版本（fommDependency），各带一个最低版本。
在依赖编辑器中，“添加组”会创建一个嵌套组，表现为一个带有各自运算符、各自条件以及“添加条件” / “添加组” / “移除组”按钮的缩进框架；条件构建器（§
8.22）会在其句子中以括号显示嵌套组，并用相同的控件编辑它们。模拟器、验证（被检验但从未被设置的标志、无法到达的步骤）、冲突检测和 Nexus
描述都能理解嵌套组。以扁平布局保存的较旧项目模板仍可加载。
同一版本中的两个较小的更改：通过沿菜单栏滑动指针打开的菜单现在会在指针离开它时关闭（通过单击打开的菜单会一直保留，直到有内容被单击）；以及用于显示项目本身所用文字——
名称和描述中的日文、韩文、中文、阿拉伯文、泰文…——
以及正在编辑的翻译所用文字的字体会被自动加载，无论界面语言如何，从而不再有方块代替那些字符的情况，包括在翻译窗口和 Nexus 窗口的语言列表中。
菜单组织为 文件、编辑、项目、工具 和 帮助，并带有一个用于日常操作的工具栏，F1
可打开本手册。组选择类型、选项类型、运算符和主题都以通俗语言的标签显示（“恰好一个（必选）”、“推荐”、“所有条件（且）”…），悬停时附有解释，选项面板的每个部分都有一个解释相应概念的 ⓘ 标记。确认对话框会使窗口变暗，接受 Enter 和
Esc，并在问题列表很长时可滚动。

8.26 编辑游戏及其分类

工具 → 游戏 / 分类… 会打开一个独立窗口，用于维护
“模组信息”选项卡中提供的游戏列表，而无需手动编辑
Categories.json。左侧窗格列出各个游戏：添加游戏、删除游戏以及 上移 / 下移
按钮可更改列表及其顺序。对于选定的游戏，编辑器会显示其标识符（唯一的内部键）、名称、Nexus slug（用于链接和 Nexus 描述）、轻量插件记录上限、基础主文件（每行一个——
游戏自身的 masters 和官方 DLC）以及 Nexus
分类（每行一个，按显示顺序排列）。输入内容会在键入时进行检查：没有标识符的游戏、重复的标识符或缺少名称都会被标记，且在所有问题修复之前 保存 始终处于禁用状态。保存
会写入文件——如果随附的 assets/data/Categories.json
可写入，则写入该文件，否则写入应用程序优先读取的按用户保存的副本——
新列表会立即生效；关闭 会在确认后放弃未保存的编辑。

8.27 树颜色

工具 → 树颜色… 可为项目树中的每种行赋予各自的颜色——
模组信息、步骤、组、选项、必需文件和条件安装——
使大型安装程序的结构一目了然，这在视力大不如前时也很有帮助（可与界面字体大小配合使用，§ 4.5）。窗口列出这六种类型，并显示色块、该颜色的图标和 RGB
值；“重置颜色”可恢复默认值。每个主题各有一套颜色：窗口编辑当前所用主题的那一套并加以说明，在深色与浅色之间切换（§ 4.5）也会切换颜色套组。颜色保存在 Config.ini 中（附录 A）。选中的行始终保持标准文本颜色，以便在选中背景上保持清晰可读。
单击色块会打开“选择颜色”窗口，即与 XIMOD Translator
中相同的颜色选择器。其“索引颜色”标签页显示 AutoCAD 颜色索引：由色相和深浅组成的 24 × 10 网格、九种标准颜色、灰度渐变，以及用于输入 1 到 255
之间索引的“颜色：”字段。其“真彩色”标签页提供色相 / 饱和度区域、亮度滑块、H、S 和 L
字段、当前颜色的红、绿、蓝值，以及可输入诸如 255,128,0 之类值的“RGB
颜色：”字段。“确定”会立即应用该颜色——树会在窗口后方重绘——并保存它；“取消”或 Esc 则保留之前的颜色。

8.28 文件对话框会记住您上次的位置

当您向选项添加来源（“添加文件”、“添加文件夹”）或浏览图像时，对话框不再每次都从项目根目录开始：它会在您上次所选文件或文件夹的父文件夹中打开。为某个选项选择 Feature D\A 后，下一个对话框会在 Feature D 中打开，B 和 C
就在眼前；在某个文件夹内选择文件后，下一个对话框会在该文件夹中打开。这一记忆在整个会话期间有效，并在切换项目时重置；如果记住的文件夹位于根目录之外或已不存在，对话框会回到根目录。对于大型安装程序，最快的方式仍是资源管理器：打开一次功能文件夹，然后将每个子文件夹拖放到其对应的选项上（§ 8.5）。

8.29 预设向导

项目 → 预设向导… 会在当前项目上构建 § 7.5 中所述的“预设 +
自定义”模式：用户可以选择一个预设后就此完成，也可以选择一个预设后仍调整几项功能，或者全部手动搭配——
而任何文件都绝不会被安装两次。手动完成这些意味着要在每个选项上设置标志、使用动态类型、设置可见性条件，并为每个选项创建一个条件安装；向导只需一步可撤销的操作即可完成所有这些。
在“从现有页面”标签页中，勾选作为功能页面的步骤（预设从这些页面的选项中进行选择），检查生成的名称——
安装模式步骤、预设组、“沿用预设”和“自定义”选项、预设标志、“无”和“无预设”选项的名称——

然后定义预设：“添加预设”会创建一列，矩阵将每个已勾选页面中的每个选项显示为一行。在互斥组（SelectExactlyOne / SelectAtMostOne）中，为每个预设选择一个选项或“（无）”；在多选组中，可勾选任意数量。底部的栏会报告第一个问题（未勾选页面、没有预设、缺少名称、两个预设同名），在问题修复之前“应用”始终处于禁用状态。
“应用”会插入安装模式步骤（放在首位，或放在上一次运行留下它的位置）：“预设”组，即在各预设与“无预设（自行搭配）”之间的单选，每个选项只写入 preset 标志；然后为每个功能页面插入一个双选项组——“沿用预设的选择”或“自定义”——只有该组写入该页面的“自定义”标志。每个功能页面仅在其“自定义”选项被选中时可见；向导向其选项添加标志，将某个预设所选的变体在该预设下标记为 Recommended，向互斥组添加一个“无”选项（默认推荐，在选择了某项内容的预设下为可选；SelectAtMostOne 组会变为 SelectExactlyOne，以免模组管理器显示第二个“无”），并将选项的文件移到“条件安装项”标签页：每个选项对应一组，当该页面已自定义且该选项在其中被勾选时，或者当该页面交由预设决定且该预设选择了此选项时安装。任何标志都绝不会由两个可以同时选中的选项写入，因此在安装程序中后退并更改任何内容时，始终会安装用户最终看到的内容（§ 7.5）。消息会总结所做的操作；编辑 → 撤销 可还原整个操作。然后用 项目 → 预览安装程序…（仅预设、预设加一个自定义页面、仅页面）和 项目 → 验证 FOMOD 检查结果，后者能够识别这种模式（§ 8.1）。
“新建骨架”标签页适用于尚无页面的项目：用页面名称、选择类型（互斥或多选）以及每行一个变体来描述每项功能，然后“创建页面”会将它们添加到项目中——不含文件——并切换到第一个标签页，同时勾选这些页面。在那里定义预设并应用，之后再分配文件：手动选择的内容分配到页面上，预设安装的内容分配到向导创建的条件安装中。
在向导已经转换过的项目上再次运行向导，不会在第一个结构之上再叠加第二个结构：窗口打开时会载入现有的页面、名称和预设，并给出相应提示。添加或修改预设，必要时勾选更多页面，然后“应用”会在一个可撤销的步骤中重建整个结构并保留标志名称，从而使模组的更新版本与其上一个版本保持兼容。由版本 2.1.0 或 2.2.1 创建的结构同样会被识别，并按当前方式重建：再次运行向导，应用，然后发布更新。

8.30 在 FOMOD 之间剪切、复制和粘贴

步骤、组和选项可以剪切、复制和粘贴：使用 编辑 → 剪切、复制和粘贴，或树中节点右键菜单里的相同命令，或在没有文本字段获得焦点时按 Ctrl+X、Ctrl+C 和 Ctrl+V——在文本字段中，这些键仍作用于文本。复制会带上节点及其包含的全部内容：组和选项、文件、图像、标志、条件以及依赖模式。剪切会将节点从项目中移除并保留在剪贴板中；编辑 → 撤销 可将其恢复。
粘贴遵循当前选择：步骤插入到所选步骤之后（未选择任何内容时插入到末尾），组放入所选步骤中，若有所选组则放在其后，选项放入所选组中，放在所选选项之后。粘贴的节点会被选中，且整个粘贴是一次可撤销的更改。当所选内容无法接收该节点时——未选择步骤时粘贴组、未选择组时粘贴选项——会有消息提示应选择什么。
剪贴板属于应用程序，而不属于某个项目：在一个 FOMOD 标签页中复制，切换到另一个标签页后粘贴即可。副本也会放到系统剪贴板中，因此在一个 XIMOD 窗口中复制的节点可以粘贴到另一个 XIMOD 窗口中。路径是相对于模组文件夹的，因此从另一个模组带来的节点可能指向本模组没有的文件：粘贴后，XIMOD 会报告其中有多少文件在本模组文件夹中缺失（随后可借助 项目 → 查找缺失文件…，§ 8.32，重新指向它们），并列出该节点所检测、但本项目中没有任何选项设置的标志——永远不会出现的页面，或类型永远不会改变的选项，都会被立即发现。

8.31 在磁盘上重命名来源

在文件资源管理器中重命名文件或文件夹，会破坏项目对它的所有引用。文件表格——
选项的“源文件”表格（§ 4.2）、“必需安装项”标签页（§ 4.3）以及条件集的文件（§ 4.4）——
中的“在磁盘上重命名…”按钮改为从 XIMOD
内重命名所选文件或文件夹，并同时更新引用：每个使用它的选项、必需文件和条件集，对于文件夹，还包括对其内部任何内容的每个引用。项目一旦有了根目录，该按钮即可使用。
窗口显示当前路径、一个用于输入新名称的字段（该项保留在同一文件夹中）以及将随之更新的引用数量；按“重命名”或 Enter 即可应用。包含 Windows 禁止的字符（如 : * ? 或
|）或以句点或空格结尾的名称、该文件夹中已被占用的名称，或已不存在的来源，都会在窗口中报告，窗口保持打开。仅更改大小写（main.esp →
Main.esp）在所有系统上均有效，并且每个引用都保留其分隔符样式（\ 或
/）。该项会立即在磁盘上重命名：编辑 → 撤销 恢复的是项目的引用，而不是磁盘上的名称。

8.32 查找缺失文件

文件也会在 XIMOD 之外被重命名和移动——
例如在重新整理模组时，或在解压其新版本之后。项目 → 查找缺失文件…
会扫描模组文件夹，列出项目使用但已不存在的每个文件、文件夹和图像，每个路径只列一次，并附上使用它的引用数量。
对于每一项，窗口会提出在模组文件夹中找到的候选项：另一个文件夹中的同名项（已移动），或在其原来所在的文件夹中、项目尚未使用的相似名称，对于文件还要求扩展名相同（已重命名）。当其他位置恰好存在一个同名项时，它会被预先选中。可以在列表中选择另一个候选项，选择“（保持不变）”以维持原样，或单击“浏览…”选取模组文件夹中的任意文件或文件夹。
“应用”会一次性更新所有选定的引用——对于文件夹，也包括其内部的引用——
作为一次可撤销的更改，然后重新扫描模组文件夹；在您自行移动文件之后，“重新扫描”执行相同的操作。与文件监视不同，这也能发现 XIMOD
关闭期间发生的变化，并且绝不会在未经您确认的情况下修改项目。项目 → 验证 FOMOD（§ 4.8）仍会报告依然缺失的文件。

附录

附录 A —— Config.ini 文件

配置保存在 Config.ini 文件中，位于可执行文件旁。以下是一个完整示例：

```
; XIMOD Architect Configuration
[General]
Locale=fra
Country=FRA
FirstStart=1
Theme=Dark
FontSize=14
ReplaceNewlines=1
MaxRecentFiles=10
SplashScreenSeconds=2

[Window]
WindowWidth=1280
WindowHeight=800

[WindowPositions]
WinPos_ximod_translation=120,80
WinSize_ximod_translation=1100,700
WinPos_ximod_preview=200,140
WinSize_ximod_preview=900,650

[Scripts]
PreSaveScript=
PostSaveScript=

[RecentFiles]
RecentFile0=C:\Mods\MyMod
```

在先前各键之外又新增了两个键：Country（所选国家/地区的 ISO 3166-1 alpha-3 代码）和 FirstStart（参见 § 2.5）。

[WindowPositions] 节会记住各自由窗口的几何信息（参见 § 2.6）。对于每个窗口，WinPos_<id> 存储其位置（x,y），WinSize_<id>
存储其大小（宽,高）。这些条目仅在用户实际移动或调整过窗口大小之后才会创建；没有它们时，窗口会居中于主窗口打开。主窗口的大小保存在 [Window] 中（WindowWidth / WindowHeight），但其位置不保存。

附录 B —— 键盘快捷键

在 macOS 上，Ctrl 键对应 Cmd。

全局快捷键

快捷键	操作
Ctrl+N	新建项目。
Ctrl+O	打开一个文件夹。
Ctrl+Shift+O	打开一个文件。
Ctrl+S	保存（生成 FOMOD）。
Ctrl+,	打开设置。
Ctrl+Q	退出应用。
F1	打开用户手册（帮助 → 用户手册）。
Ctrl+X / Ctrl+C / Ctrl+V	剪切、复制或粘贴树中所选的步骤、组或选项（§ 8.30）。在文本字段中，这些键作用于文本。

窗口与对话框

快捷键	操作
Esc	关闭活动的自由窗口（国家/地区选择、属性、XML 编辑器）或模态对话框（设置、关于、确认、验证报告、脚本）。对于确认对话框，作用为“取消”。对翻译编辑器和主窗口无效。

列表导航 —— 属性与翻译编辑器

键	操作
↑ / ↓	将选择上移 / 下移一行。
Page Up / Page Down	滚动一屏；步长会适应窗口高度。
Home / End	跳到第一行 / 最后一行（最后一行会被完全带入视野）。
Enter	属性：选中高亮的项。翻译编辑器：将光标置于该行的输入字段中。
Tab / Shift+Tab	属性：下一行 / 上一行（永远不会落到旗帜上）。翻译编辑器（编辑时）：下一个 / 上一个翻译字段。

旗帜网格 —— “国家/地区选择”窗口

键	操作
↑ ↓ ← →（或 8 2 4 6）	在网格内移动选择框。
Page Up / Page Down	滚动一屏旗帜。
Home / End	跳到第一面 / 最后一面旗帜（最后一行会被完全带入视野）。
Tab / Shift+Tab	下一面 / 上一面旗帜。
Enter	确认高亮的国家/地区。
Esc	关闭窗口。

附录 C —— 术语表

术语	定义
FOMOD	基于 info.xml 和 ModuleConfig.xml 的模组安装程序格式；它是 Bethesda 游戏的标准格式，Vortex 和 Mod Organizer 2 也支持将其用于大多数其他游戏。
步骤（Step）	安装向导的页面，包含一个或多个组。
组（Group）	共享一种选择类型的选项（plugins）的集合。
Plugin	提供给用户的选项，与要安装的文件相关联。
标志（Flag）	用于驱动安装的条件标志（名称 = 值 对）。
本族名（Endonym）	一个国家/地区或语言用该语言书写的名称。
ISO 639-3	为翻译文件夹命名的三字母语言代码（例如 fra、eng）。
ISO 3166-1 alpha-3	三字母国家/地区代码（例如 FRA、USA）。
根目录	包含模组文件和所生成的 fomod 子文件夹的工作文件夹。

附录 D —— 完整 FOMOD 示例

本附录呈现一个虚构但完整的项目——“Dawnlight”，一个 Skyrim Special Edition 的光照模组——它实践了 XIMOD Architect 的全部可能性：必装文件、多个步骤、一个步骤可见性条件、五种组类型、五种选项类型、条件标志、动态选项类型（依赖模式）以及条件安装。下面的两个文件构成单个项目，并按 XIMOD 生成它们的样子重现（缩进已简化）。

D.1 info.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<fomod>
    <Name>Dawnlight</Name>
    <Author>Patrice</Author>
    <Version>1.2.0</Version>
    <Website>https://www.nexusmods.com/skyrimspecialedition/mods/98765</Website>
    <Description>Improved lighting, candles and weather for Skyrim SE.</Description>
    <Groups>Visuals and Graphics</Groups>
    <Game>skyrimSpecialEdition</Game>
</fomod>
```

D.2 ModuleConfig.xml

示例所涵盖的功能：

- 必装文件：requiredInstallFiles（一个文件和一个文件夹）。
- 步骤 1：SelectAll 组（选项 Required）和 SelectExactlyOne（选项 Recommended / Optional，res 标志）。
- 步骤 2：SelectAny 组（candles 和 moon 标志）和 SelectAtMostOne。
- 步骤 3：可见性条件（<visible>），然后是一个 SelectAtLeastOne 组，其中有一个动态类型选项（dependencyType：若 candles=On 则为 Recommended，若某个文件缺失则为 NotUsable）和一个 CouldBeUsable 选项。
- 条件安装：两个模式（And 和 Or 运算符，flag 和 file 依赖，Active / Inactive / Missing 状态）。

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>Dawnlight</moduleName>
  <moduleImage path="fomod\header.png"/>
  <requiredInstallFiles>
    <file source="Dawnlight.esp" destination="Dawnlight.esp" priority="0"/>
    <folder source="core\meshes" destination="meshes" priority="0"/>
  </requiredInstallFiles>
  <installSteps order="Explicit">
    <installStep name="Main installation">
      <optionalFileGroups>
        <group name="Core module" type="SelectAll">
          <plugins order="Explicit">
            <plugin name="Base files">
              <description>Scripts and meshes essential to the mod.</description>
              <files>
                <folder source="core\scripts" destination="scripts" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Required"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
        <group name="Texture resolution" type="SelectExactlyOne">
```

```
        <plugins order="Explicit">
          <plugin name="2K">
            <description>2K textures, good quality/performance balance.</description>
            <image path="fomod\2k.png"/>
            <conditionFlags>
              <flag name="res">2K</flag>
            </conditionFlags>
            <files>
              <folder source="tex\2k" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Recommended"/>
            </typeDescriptor>
          </plugin>
          <plugin name="4K">
            <description>4K textures, for powerful setups.</description>
            <image path="fomod\4k.png"/>
            <conditionFlags>
              <flag name="res">4K</flag>
            </conditionFlags>
            <files>
              <folder source="tex\4k" destination="textures" priority="1"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
  <installStep name="Ambience options">
    <optionalFileGroups>
      <group name="Lighting effects" type="SelectAny">
        <plugins order="Explicit">
          <plugin name="Dynamic candles">
            <description>Adds candles with flickering light.</description>
            <conditionFlags>
              <flag name="candles">On</flag>
            </conditionFlags>
            <files>
              <folder source="opt\candles" destination="meshes" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
          <plugin name="Enhanced moon">
            <description>Increases the intensity of the moonlight.</description>
            <conditionFlags>
              <flag name="moon">On</flag>
            </conditionFlags>
            <files>
              <folder source="opt\moon" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
      <group name="Sound ambience" type="SelectAtMostOne">
        <plugins order="Explicit">
          <plugin name="Night sounds">
            <description>Adds a nighttime sound ambience.</description>
            <files>
              <folder source="opt\sound" destination="sound" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
  <installStep name="ENB compatibility">
```

```
    <visible>
      <dependencies operator="And">
        <flagDependency flag="res" value="4K"/>
        <fileDependency file="KnownConflict.esp" state="Inactive"/>
      </dependencies>
    </visible>
    <optionalFileGroups>
      <group name="ENB preset" type="SelectAtLeastOne">
        <plugins order="Explicit">
          <plugin name="Light ENB">
            <description>Low-cost ENB preset.</description>
            <files>
              <file source="enb\light.ini" destination="enblocal.ini" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
          <plugin name="Ultra ENB">
            <description>High-end ENB preset (dynamic type).</description>
            <files>
              <file source="enb\ultra.ini" destination="enblocal.ini" priority="1"/>
            </files>
            <typeDescriptor>
              <dependencyType>
                <defaultType name="Optional"/>
                <patterns>
                  <pattern>
                    <dependencies operator="And">
                      <flagDependency flag="candles" value="On"/>
                    </dependencies>
                    <type name="Recommended"/>
                  </pattern>
                  <pattern>
                    <dependencies operator="Or">
                      <fileDependency file="EnbHelper.dll" state="Missing"/>
                    </dependencies>
                    <type name="NotUsable"/>
                  </pattern>
                </patterns>
              </dependencyType>
            </typeDescriptor>
          </plugin>
          <plugin name="Experimental extension">
            <description>Additional effects, not guaranteed.</description>
            <files>
              <folder source="enb\extra" destination="textures" priority="2"/>
            </files>
            <typeDescriptor>
              <type name="CouldBeUsable"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
</installSteps>
<conditionalFileInstalls>
  <patterns>
    <pattern>
      <dependencies operator="And">
        <flagDependency flag="res" value="4K"/>
        <flagDependency flag="candles" value="On"/>
      </dependencies>
      <files>
        <folder source="cond\candles4k" destination="textures" priority="0"/>
      </files>
    </pattern>
    <pattern>
      <dependencies operator="Or">
        <flagDependency flag="moon" value="On"/>
        <fileDependency file="SkyUI_SE.esp" state="Active"/>
      </dependencies>
      <files>
        <file source="cond\patch.esp" destination="patch.esp" priority="0"/>
      </files>
    </pattern>
```

```
    </patterns>
  </conditionalFileInstalls>
</config>
```

附录 E —— 许可证与致谢

XIMOD Architect 的源代码在 MIT 许可证下分发（© 2025-2026）。以下第三方组件各自保留其自身的许可证；完整详情和归属声明见随应用提供的 THIRD-PARTY-NOTICES 文件。

组件	来源	许可证
XIMOD Architect 代码	—	MIT
Noto 字体	Google Fonts / Noto 项目	SIL Open Font License 1.1
旗帜（SVG）	drapeauxdespays.fr	免版税（含商业用途）
ISO 3166-1 alpha-3 国家/地区代码	ISO —— iso.org/obp	免费获取
ISO 639-3 语言代码	SIL —— iso639-3.sil.org	需要署名（见下文）
Rust 库	crates.io 生态系统	MIT / Apache-2.0（多数）

ISO 639-3 署名：语言代码来自 iso639-3.sil.org（来源须予以致谢）。XIMOD 添加的数据——本族名、关联字体、国家/地区 ↔ 语言 映射——不属于 ISO 639-3 标准的一部分，且该标准的标识符未被修改。此外，本应用不得提供任何再分发全套代码的手段。